

Passwords

[Passwords.app]

Yes. With your feature vector and stack, the clean architecture is to treat **latitude + longitude as the starting query**, not as the complete input to LightGBM.

The model should behave more like:

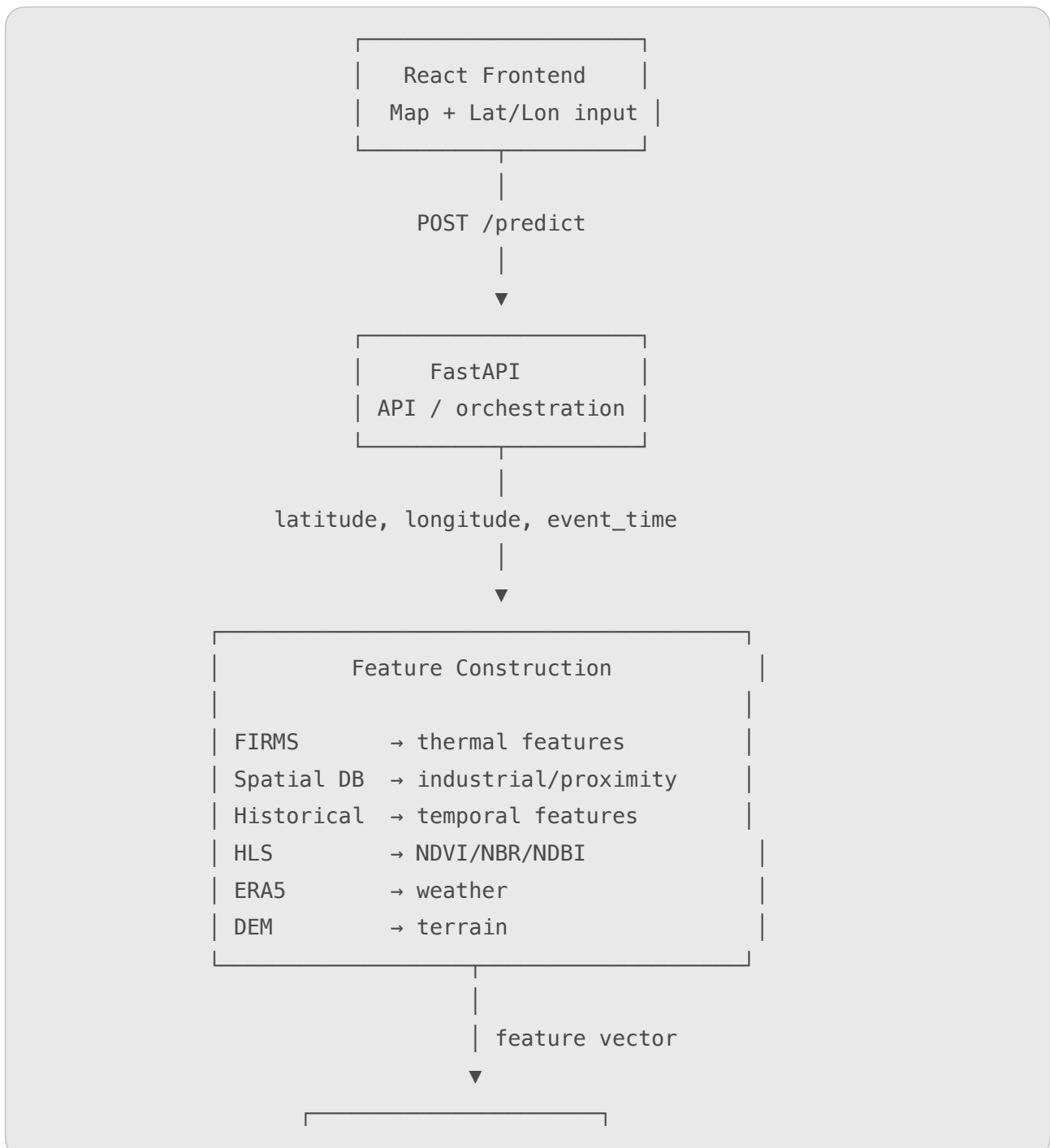
Lat/Lon → identify spatial/temporal context → construct 37-ish features → LightGBM → classification → store result → GIS visualization

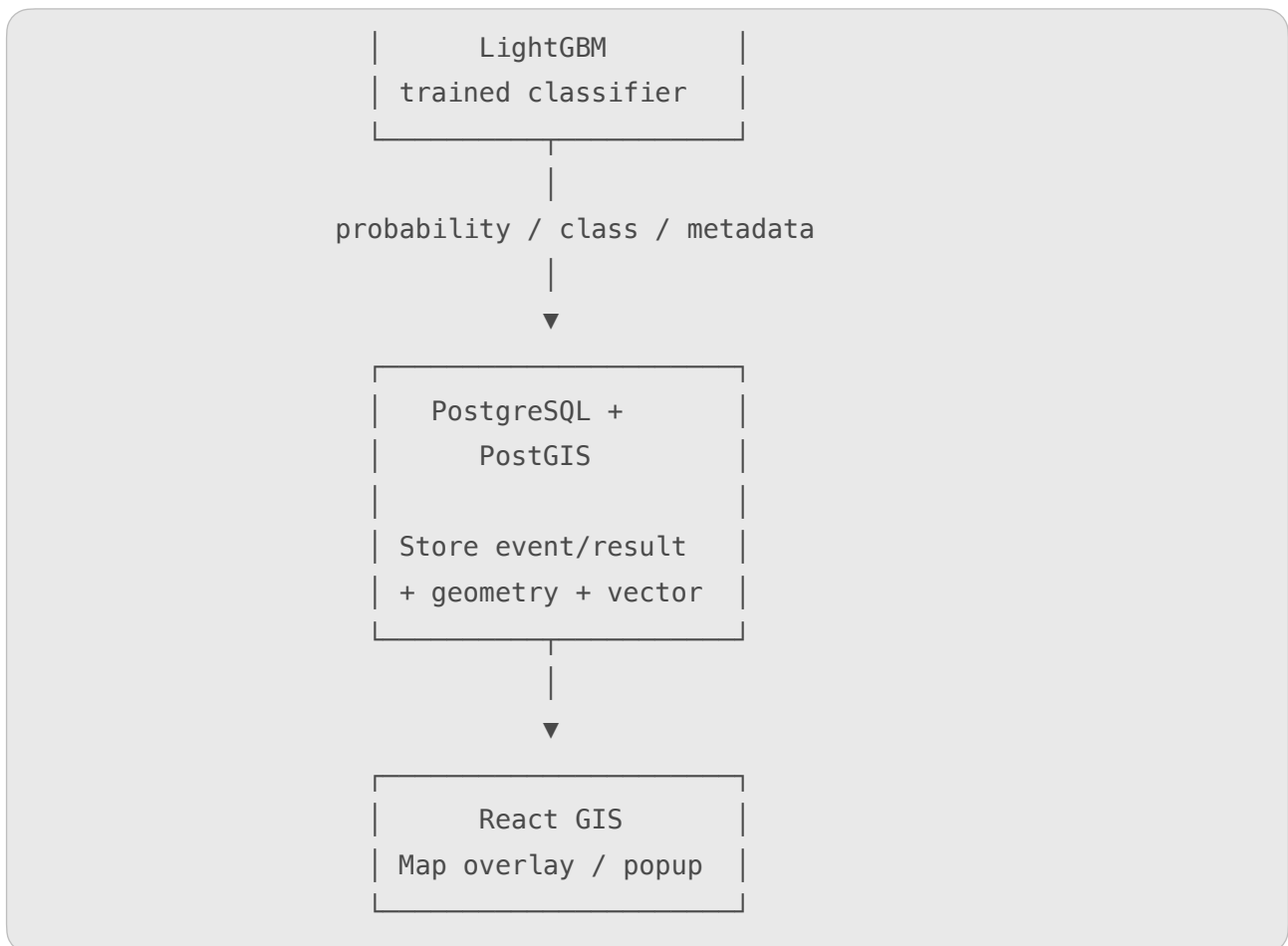
rather than:

Lat/Lon → LightGBM → result

That distinction is very important for your project.

1. Your overall architecture





The most important database design principle is:

Do not store only the final classification. Store enough information to reconstruct why the model produced that classification.

That is especially important for a research/engineering project like this.

2. First, separate your data into different categories

Your database should not be one giant table containing everything.

I would divide your PostgreSQL/PostGIS database roughly into:

1. thermal_events
2. industrial_facilities
3. land_cover
4. historical_thermal_events
5. satellite_imagery_metadata
6. weather_data
7. terrain_data
8. feature_vectors
9. model_predictions
10. model_registry

You may combine some of these depending on how much data you actually ingest.

The distinction between **raw/source data**, **derived features**, and **model outputs** is extremely important.

3. thermal_events — your starting point

This is probably the most important table.

Every thermal anomaly detected by FIRMS becomes an event/observation.

Conceptually:

```
thermal_events

event_id
geometry
latitude
longitude
acquisition_time
sensor
satellite
frp
brightness_temperature
confidence
day_night
...
```

For example:

```
event_id      = 182734
latitude      = 19.8765
longitude     = 73.4567
acquisition_time = 2026-08-21 13:42:00
frp           = 42.7
brightness_temperature = 341.2
confidence    = high
sensor        = VIIRS
geometry      = POINT(...)
```

Why store this?

Because this is the **original observation**.

Your ML pipeline should ideally be able to say:

"This prediction came from thermal observation X."

rather than simply:

"Lat 19.8765, Lon 73.4567 was classified as industrial."

4. Use PostGIS geometry

Don't rely exclusively on:

```
latitude
longitude
```

Store a PostGIS geometry as well.

For example conceptually:

```
geometry geometry(Point, 4326)
```

So a point becomes:

```
POINT(73.4567 19.8765)
```

Notice:

```
POINT(longitude latitude)
```

not latitude-longitude.

This gives you spatial operations such as:

```
ST_DWithin()  
ST_Distance()  
ST_Intersects()  
ST_Contains()  
ST_Within()
```

These become extremely useful for your features.

For example:

```
distance_to_refinery
```

can be calculated spatially.

5. Industrial facilities table

You need another major table:

```
industrial_facilities
```

Something like:

```
facility_id  
facility_name  
facility_type  
geometry  
latitude  
longitude  
source  
country  
state  
district  
active_status
```

facility_type could contain categories such as:

```
REFINERY  
OIL_GAS
```

```
POWER_PLANT
STEEL
CEMENT
MINE
LANDFILL
LNG
...
```

The geometry could be either:

```
POINT
```

or

```
POLYGON
```

depending on your source data.

6. Why industrial facilities need to be stored

Look at your features:

```
distance_to_refinery
distance_to_oil_gas_facility
distance_to_power_plant
distance_to_steel_cement_plant
distance_to_mine
distance_to_landfill
distance_to_nearest_industrial_structure
in_industrial_polygon
industrial_facility_count_3km
```

These are **not properties of the thermal pixel itself**.

They are derived by spatial queries.

Suppose the user gives:

```
lat = 19.8765
lon = 73.4567
```

Your backend creates:

```
POINT(73.4567 19.8765)
```

Then PostGIS can search:

```
"What industrial facilities are around this point?"
```

For example:

```
nearest refinery = 1.8 km
```

```
nearest power plant = 6.4 km
nearest mine = 12.1 km
industrial facilities within 3 km = 4
```

Those numbers become LightGBM features.

7. Land-cover database

You also need to store or have access to land-cover information.
Conceptually:

```
land_cover

cell_id
geometry
land_cover_class
source
resolution
valid_from
valid_to
```

For example:

```
geometry      → polygon
land_cover_class → INDUSTRIAL
```

or:

```
FOREST
CROPLAND
WATER
URBAN
BARE_SOIL
GRASSLAND
...
```

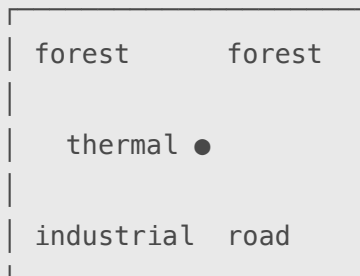
This feeds:

```
land_cover_class
cropland_fraction_1km
forest_fraction_1km
industrial_fraction_1km
bare_soil_fraction_1km
```

8. Important distinction: raw land cover vs derived fractions

Suppose your event occurs here:

1 km neighborhood



You might calculate:

```
forest_fraction_1km = 0.38
industrial_fraction_1km = 0.31
cropland_fraction_1km = 0.17
bare_soil_fraction_1km = 0.14
```

These values become part of your feature vector.
The raw land-cover data can remain in its own table/data store.

9. Historical thermal events

This table is extremely important for your temporal features.
You need to retain previous observations rather than only the latest event.
For example:

```
thermal_events
-----
event_id
geometry
acquisition_time
frp
bt_fire
confidence
sensor
...
```

Then for a new event:

2026-09-21

you can search backward:

previous 30 days

```
previous 90 days  
historical observations
```

and calculate:

```
historical_frp_mean  
historical_frp_std  
detection_frequency_30d  
detection_frequency_90d  
persistence_90d  
active_year_fraction  
active_month_fraction  
night_detection_fraction  
days_since_previous_detection  
time_since_first_detection
```

10. Why temporal history is so important

Imagine two points.

Point A

```
detected:  
2026-09-21
```

Only once.

Point B

```
detected:  
June 1  
June 7  
June 14  
June 22  
July 3  
July 9  
July 17  
August 1  
August 8  
August 19  
September 2  
September 21
```

The instantaneous thermal measurement could be similar.
But the temporal behavior is radically different.
Your feature vector captures this through:

```
detection_frequency_30d
detection_frequency_90d
persistence_90d
active_month_fraction
...
```

This is one of the strongest reasons **not to make your model purely a thermal classifier**.

11. Your historical features are calculated, not manually stored

For example:

```
historical_frp_mean
```

could be calculated from previous observations.
Conceptually:

```
previous FRP:
```

```
20
25
22
30
24
```

Then:

```
mean = 24.2
```

Likewise:

```
historical_frp_std
```

is derived from historical observations.
And:

```
robust_frp_zscore
```

could be based on robust statistics such as median/MAD rather than ordinary mean/std.
The exact formula should be fixed during feature-engineering design and used identically during training and inference.

12. HLS imagery

You have:

```
ndvi_pre
nbr_pre
ndbi_pre
builtup_fraction_pre
```

```
optical_valid_fraction  
hls_days_before_event  
data_availability_flag
```

You need metadata about the imagery used.
For example:

```
satellite_imagery  
  
image_id  
source  
acquisition_time  
geometry  
cloud_cover  
processing_level  
resolution  
asset/reference
```

Then your feature pipeline finds the appropriate **pre-event image**.
Suppose event:

```
2026-09-21
```

and you choose a pre-event image:

```
2026-09-17
```

Then:

```
hls_days_before_event = 4
```

13. Why `hls_days_before_event` should be stored

Suppose:

```
NDVI = 0.12
```

That number alone doesn't tell you whether the imagery was:

```
2 days before
```

or:

```
45 days before
```

The temporal distance affects interpretation.
Therefore:

```
hls_days_before_event
```

is part of the feature vector.

14. Weather table

You need weather information corresponding to the event.
Conceptually:

```
weather_observations

observation_time
geometry/grid_cell
wind_speed
wind_direction
vpd
rainfall
...
```

Your model features are:

```
wind_speed
wind_direction_sin
wind_direction_cos
vpd
rainfall_72h
```

15. Why encode wind direction using sin/cos?

This is an important detail.
Suppose wind direction is:

```
359°
```

and:

```
1°
```

Numerically they appear far apart:

```
359 - 1 = 358°
```

but physically they are almost identical directions.
So instead of directly feeding:

```
wind_direction = 359
```

you encode:

```
sin(direction)
cos(direction)
```

This gives the model a continuous circular representation.

Your database can store the original direction if available, while your **feature table stores the transformed model inputs**.

16. Terrain

You have:

```
elevation
slope
```

These can come from DEM-derived data.

You don't necessarily need a huge terrain table if you're generating these values from a raster/terrain service.

But the inference system needs access to the source.

For reproducibility, record something like:

```
terrain_source
terrain_resolution
```

in your metadata/configuration.

17. Now comes the most important table: **feature_vectors**

I strongly recommend having a table representing the actual vector sent into LightGBM.

Conceptually:

```
feature_vectors
```

with:

```
feature_vector_id
event_id
created_at
feature_version

frp_log1p
frp_density
bt_fire
bt_delta
confidence
day_night
sensor_agreement

cluster_area
cluster_compactness
centroid_shift

historical_frp_mean
historical_frp_std
robust_frp_zscore
```

```
...  
  
distance_to_refinery  
...  
  
land_cover_class  
...  
  
ndvi_pre  
...  
  
wind_speed  
...  
  
elevation  
slope
```

You have roughly **40 features** in your vector.
The exact number depends on how you encode categorical fields such as:

```
land_cover_class  
day_night  
confidence
```

18. Why store the feature vector?

This is crucial.
Imagine today the model says:

```
industrial_fire_probability = 0.91
```

Six months later you retrain your model and it says:

```
industrial_fire_probability = 0.68
```

If you stored only the final result, debugging becomes difficult.
But if you stored:

```
feature_vector_id = 92831  
feature_version = v3
```

and all feature values, you can inspect:

```
FRP  
distance to refinery  
forest fraction  
historical persistence
```

```
NDVI
rainfall
...
```

and understand exactly what the model saw.

19. Don't mix raw data and model features unnecessarily

A useful architecture is:

```
RAW
↓
DERIVED
↓
MODEL INPUT
↓
MODEL OUTPUT
```

For example:

```
FIRMS raw observation
      ↓
thermal_events
      ↓
feature engineering
      ↓
feature_vectors
      ↓
LightGBM
      ↓
model_predictions
```

This separation makes the system much easier to debug.

20. Model registry

You should also have:

```
model_registry
```

Something like:

```
model_id
model_name
model_version
training_date
feature_version
```

```
training_dataset_version
model_file_path
threshold
created_at
metrics_json
```

For example:

```
model_id = 7
model_version = "lgbm_v3"
feature_version = "features_v5"
training_date = 2026-09-15
model_file = ...
```

The actual .txt/.pkl/serialized LightGBM model doesn't necessarily need to be stored directly inside PostgreSQL.
You can store the model in object/file storage and keep its metadata/path/hash in PostgreSQL.

21. Prediction table

Finally:

```
model_predictions
```

Something like:

```
prediction_id
event_id
feature_vector_id
model_id

predicted_class
industrial_probability
natural_probability

prediction_time
decision_threshold

geometry
```

For example:

```
prediction_id = 500183

event_id = 182734

model_version = lgbm_v3
```

```
predicted_class = INDUSTRIAL_FIRE

industrial_probability = 0.91
natural_probability = 0.09

prediction_time = ...
```

This is what your React GIS interface primarily consumes.

22. Now let's trace your actual request

This is the part you specifically asked for.
Suppose the React user clicks:

```
Latitude:
19.8765

Longitude:
73.4567
```

They press:

```
Analyze
```

React sends something like:

```
{
  "latitude": 19.8765,
  "longitude": 73.4567
}
```

to:

```
POST /predict
```

23. FastAPI receives it

FastAPI validates:

```
latitude ∈ [-90, 90]
longitude ∈ [-180, 180]
```

Then converts:

```
lat = 19.8765
lon = 73.4567
```

into a PostGIS point:

```
POINT(73.4567 19.8765)
```

At this point, **you still don't have the LightGBM input vector**.
You only have the spatial location.

24. Step 1 — Find the thermal event

The backend searches your thermal-event database.
Conceptually:

```
Find thermal observations near  
POINT(73.4567 19.8765)
```

You need a defined spatial tolerance.
For example, conceptually:

```
within X meters
```

and possibly a time/event-selection rule.

This is an architectural decision you need to define explicitly.

If the user selects an existing thermal anomaly from your map, the cleanest API is actually:

```
{  
  "event_id": 182734  
}
```

rather than relying entirely on lat/lon.
You can still accept lat/lon for convenience.

25. Step 2 — retrieve the thermal features

Suppose the selected event contains:

```
FRP = 42.7  
BT = 341.2  
confidence = 0.86
```

Feature engineering produces:

```
frp_log1p  
frp_density  
bt_fire  
bt_delta  
confidence  
day_night  
sensor_agreement
```

For example:

```
frp_log1p = log(1 + 42.7)
```

The exact transformation must be the same transformation used during model training.

26. Step 3 — identify the thermal cluster

You then determine which detections belong to the same spatial cluster/event.
Suppose:

```
12 thermal pixels
```

are spatially grouped.
You derive:

```
cluster_area  
cluster_compactness  
centroid_shift
```

Conceptually:

```
individual thermal pixels  
    ↓  
spatial grouping  
    ↓  
cluster  
    ↓  
cluster statistics
```

For example:

```
cluster_area = 2.8 km2  
cluster_compactness = ...  
centroid_shift = ...
```

27. Step 4 — calculate historical behavior

The backend looks backward from the event.
For example:

```
2026-09-21  
    ↑  
    |  
90-day history  
    |  
2026-06-23
```

It retrieves previous thermal observations associated with that location/cluster.
Suppose it finds:

```
35 historical detections
```

Then calculate:

```
historical_frp_mean
historical_frp_std
robust_frp_zscore

detection_frequency_30d
detection_frequency_90d
persistence_90d

active_year_fraction
active_month_fraction

night_detection_fraction

days_since_previous_detection
time_since_first_detection
```

Now the model knows not just:

"There is heat here."

but also:

"What has historically happened at this location?"

28. Step 5 — PostGIS performs proximity analysis

Now the point:

```
POINT(73.4567 19.8765)
```

is spatially compared against your industrial database.

PostGIS determines:

```
nearest refinery
nearest oil/gas facility
nearest power plant
nearest steel/cement plant
nearest mine
nearest landfill
nearest industrial structure
```

Suppose:

```
distance_to_refinery = 1.4 km
distance_to_oil_gas_facility = 3.7 km
distance_to_power_plant = 11.2 km
distance_to_steel_cement_plant = 7.8 km
distance_to_mine = 18.4 km
distance_to_landfill = 4.2 km
distance_to_nearest_industrial_structure = 0.8 km
```

and:

```
in_industrial_polygon = 1
```

and:

```
industrial_facility_count_3km = 5
```

Those become features.

29. Step 6 — calculate surroundings

Now create a spatial buffer around the event.

For example:

```
1 km radius
```

Then intersect that area with your land-cover dataset.

You might obtain:

```
forest = 18%  
cropland = 7%  
industrial = 62%  
bare soil = 13%
```

Therefore:

```
forest_fraction_1km = 0.18  
cropland_fraction_1km = 0.07  
industrial_fraction_1km = 0.62  
bare_soil_fraction_1km = 0.13
```

And:

```
land_cover_class = INDUSTRIAL
```

30. Step 7 — HLS extraction

Now your backend determines the appropriate **pre-event HLS image**.

Suppose:

```
event = 21 September
```

and the usable image is:

```
17 September
```

It extracts pixels around the event.

From those pixels you calculate:

```
NDVI  
NBR
```

```
NDBI
built-up fraction
optical valid fraction
```

giving something like:

```
ndvi_pre = 0.21
nbr_pre = 0.08
ndbi_pre = 0.54
builtup_fraction_pre = 0.72
optical_valid_fraction = 0.91
hls_days_before_event = 4
data_availability_flag = 1
```

31. Step 8 — weather extraction

Now find the ERA5 grid cell/time window corresponding to the event.
Suppose:

```
wind_speed = 7.2 m/s
wind_direction = 240°
VPD = ...
rainfall_72h = ...
```

You don't necessarily send raw:

```
240°
```

to LightGBM.
Instead:

```
wind_direction_sin
wind_direction_cos
```

are calculated.
So the feature vector contains the representation used during training.

32. Step 9 — terrain

Extract:

```
elevation
slope
```

at the event location.
For example:

```
elevation = 413 m
slope = 4.7°
```

33. Now you finally have the LightGBM vector

Conceptually:

```
X =  
[  
    frp_log1p,  
    frp_density,  
    bt_fire,  
    bt_delta,  
    confidence,  
    day_night,  
    sensor_agreement,  
  
    cluster_area,  
    cluster_compactness,  
    centroid_shift,  
  
    historical_frp_mean,  
    historical_frp_std,  
    robust_frp_zscore,  
    detection_frequency_30d,  
    detection_frequency_90d,  
    persistence_90d,  
    active_year_fraction,  
    active_month_fraction,  
    night_detection_fraction,  
    days_since_previous_detection,  
    time_since_first_detection,  
  
    distance_to_refinery,  
    distance_to_oil_gas_facility,  
    distance_to_power_plant,  
    distance_to_steel_cement_plant,  
    distance_to_mine,  
    distance_to_landfill,  
    distance_to_nearest_industrial_structure,  
    in_industrial_polygon,  
    industrial_facility_count_3km,  
  
    land_cover_class,  
    cropland_fraction_1km,  
    forest_fraction_1km,
```

```

    industrial_fraction_1km,
    bare_soil_fraction_1km,

    ndvi_pre,
    nbr_pre,
    ndbi_pre,
    builtup_fraction_pre,
    optical_valid_fraction,
    hls_days_before_event,
    data_availability_flag,

    wind_speed,
    wind_direction_sin,
    wind_direction_cos,
    vpd,
    rainfall_72h,

    elevation,
    slope
]

```

The exact dimensionality depends on how your categorical variables are encoded.

34. Very important: feature ordering

This is one of the easiest ways to accidentally break an ML system. Suppose during training LightGBM receives:

```

feature[0] = frp_log1p
feature[1] = frp_density
feature[2] = bt_fire
...

```

During inference you **must not accidentally do**:

```

feature[0] = elevation
feature[1] = frp_log1p
...

```

The model doesn't understand feature names conceptually at the mathematical level of the trained tree structure; it relies on the feature representation it was trained against. Therefore maintain a fixed:

```

feature_schema

```

such as:

```

FEATURE_VERSION = "v1"

```

```
0 frp_log1p
1 frp_density
2 bt_fire
3 bt_delta
4 confidence
...
```

Store this version with every feature vector and prediction.

35. What LightGBM actually does

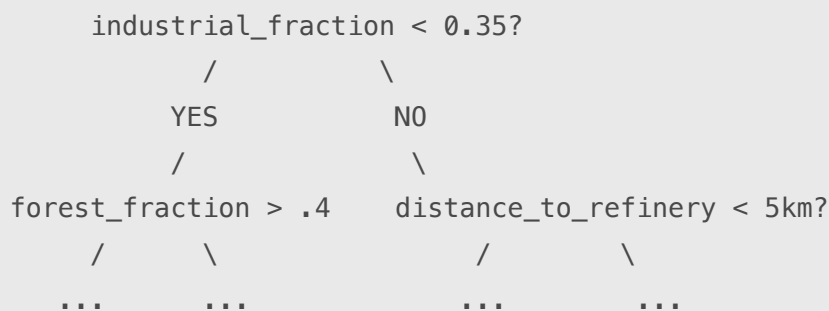
This is where your system transitions from **feature engineering** to **machine learning inference**. LightGBM is a gradient-boosted decision-tree model. Conceptually, your vector:

X

enters the model.

The model contains many decision trees.

A simplified tree might contain logic such as:



Your feature vector traverses branches.

For example:

industrial_fraction = 0.62

means:

0.62 < 0.35 ?

NO

so it follows one branch.

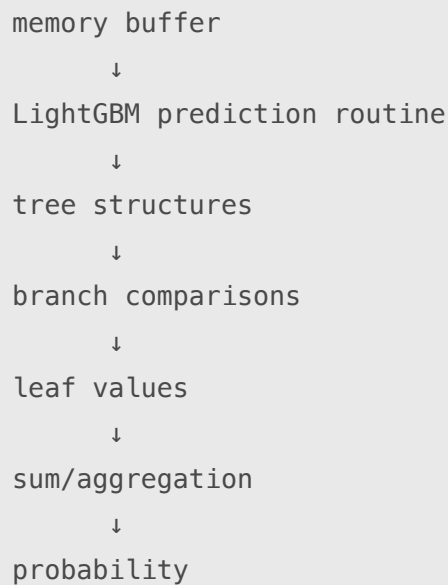
Then another tree examines another feature.

This happens across many trees.

36. Binary-level perspective of LightGBM

At the software level, your Python process has:

feature vector
↓



A feature such as:

```
frp_log1p = 3.77
```

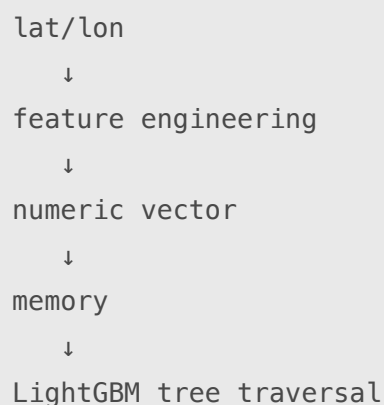
is represented in memory as a floating-point value, typically a 32-bit or 64-bit IEEE-754 value depending on the pipeline/model representation.

Conceptually a 32-bit floating-point number is:

sign	exponent	fraction
1 bit	8 bits	23 bits

But you generally **do not manually manipulate those bits** in your application. NumPy/Python/LightGBM handle the memory representation.

The important architectural point is:



37. LightGBM does not calculate your geographic features

This distinction is critical.

LightGBM does **not** know what:

```
distance_to_refinery
```

means.

It doesn't query PostGIS.

It doesn't download HLS.

It doesn't query ERA5.

It doesn't calculate NDVI.

Your FastAPI feature-engineering layer does that.

The division is:

```
PostGIS / raster / APIs
```

↓

```
feature engineering
```

↓

```
X vector
```

↓

```
LightGBM
```

↓

```
prediction
```

38. What the model outputs

Suppose your model is binary:

```
0 = non-industrial / natural
```

```
1 = industrial
```

Then LightGBM may produce something like:

```
P(class = 1) = 0.91
```

So:

```
industrial_probability = 0.91
```

and perhaps:

```
non_industrial_probability = 0.09
```

If your decision threshold is:

```
0.50
```

then:

```
0.91 >= 0.50
```

produces:

```
predicted_class = INDUSTRIAL
```

But keep the **probability and threshold separately** in your database. Don't store only the final label.

39. Prediction gets stored

Now your backend writes:

```
model_predictions
```

Something like:

```
prediction_id: 500183
event_id: 182734
feature_vector_id: 92831
model_id: 7

predicted_class: INDUSTRIAL_FIRE
industrial_probability: 0.91
threshold: 0.50

prediction_time: ...
```

And importantly:

```
geometry = POINT(73.4567 19.8765)
```

40. FastAPI returns the result

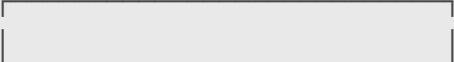
React gets something like:

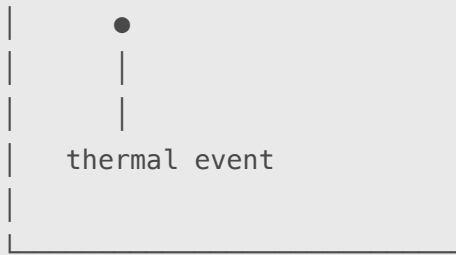
```
{
  "event_id": 182734,
  "latitude": 19.8765,
  "longitude": 73.4567,
  "classification": "INDUSTRIAL_FIRE",
  "industrial_probability": 0.91,
  "model_version": "lgbm_v3",
  "feature_version": "v5"
}
```

41. React puts it on the map

Now your GIS UI can render:

MAP





The point can be styled according to classification.
For example, conceptually:

INDUSTRIAL_FIRE

↓

red/orange marker

NON_INDUSTRIAL

↓

another marker style

The popup can show:

Classification: Industrial fire

Probability: 91%

FRP: 42.7 MW

Historical detections: 35

Nearest refinery: 1.4 km

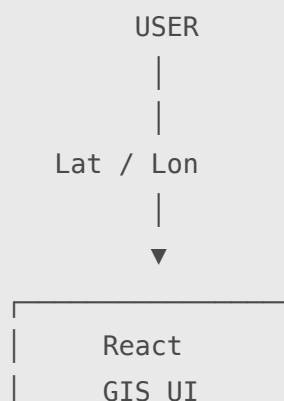
Industrial fraction: 62%

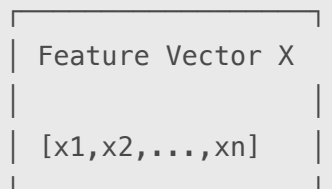
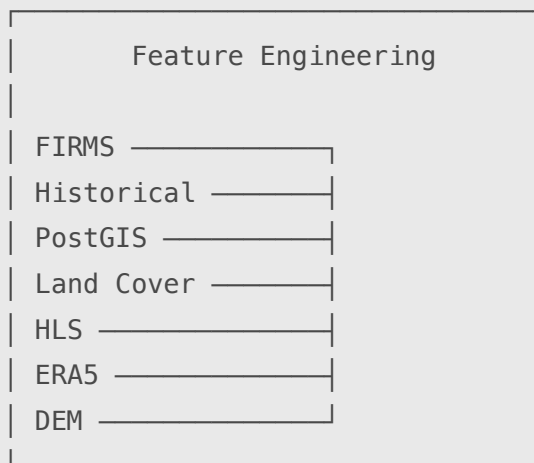
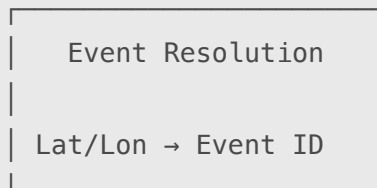
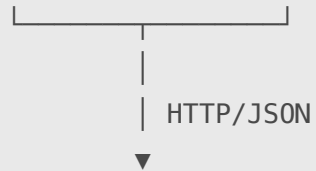
Model: lgbm_v3

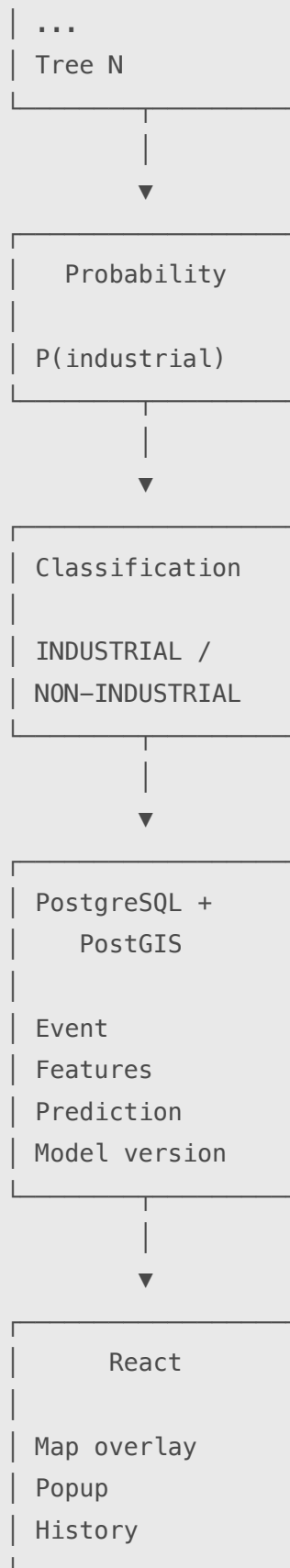
Feature version: v5

42. The complete workflow in one diagram

This is the architecture I would keep in your project documentation:







43. What I would actually store

A practical first database design would therefore look like this:

Table	Stores	Why
-------	--------	-----

thermal_events	FIRMS observations	Original thermal evidence
industrial_facilities	Refineries, plants, mines, etc.	Spatial proximity features
land_cover	Land-cover geometries/ classes	Surrounding-land features
satellite_imagery	HLS image metadata/ references	Reproducibility + imagery lookup
weather_observations	ERA5-derived values	Weather features
terrain	DEM/terrain metadata or derived values	Elevation/slope
feature_vectors	Exact ML input	Reproducibility/debugging
model_predictions	Classification/probability	Application output
model_registry	Model versions/configuration	Version tracking

You don't necessarily need to physically duplicate every raw satellite/weather pixel in PostgreSQL. For large raster datasets, keeping the actual raster/object data externally and storing **metadata, identifiers, paths/URLs, timestamps, and derived values** is usually more practical.

44. One particularly important change I'd make to your API

Instead of only:

```
POST /predict
```

with:

```
{
  "latitude": 19.8765,
  "longitude": 73.4567
}
```

I'd design the system around an **event ID**, while still allowing lat/lon.
For example:

```
POST /events/{event_id}/predict
```

or:

```
{
  "event_id": 182734
}
```

Why?

Because a coordinate doesn't uniquely identify a thermal observation.
The same location can have:

```
Event A → June 1
Event B → June 17
```

Event C → July 4

Event D → September 21

The location is the same, but the:

FRP

weather

imagery

historical state

cluster

can all be different.

So conceptually:

LOCATION

|

|— Event 1

|— Event 2

|— Event 3

|— Event 4

That is a much better data model for your problem.

45. One more important point about your current feature vector

Your feature set is fundamentally mixing **three different kinds of information**:

A. Instantaneous/event features

↓

FRP

BT

confidence

weather

imagery

B. Spatial/contextual features

↓

refinery distance

industrial fraction

land cover

terrain

C. Historical/behavioral features

↓

detection frequency

persistence
active months
historical FRP

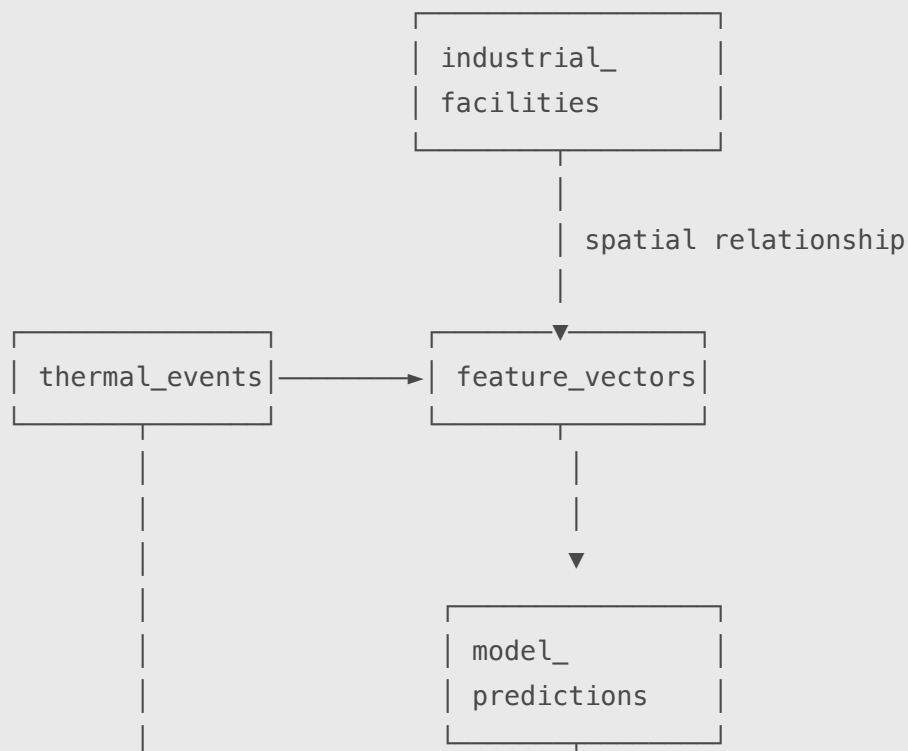
That's a useful conceptual decomposition for your project presentation:

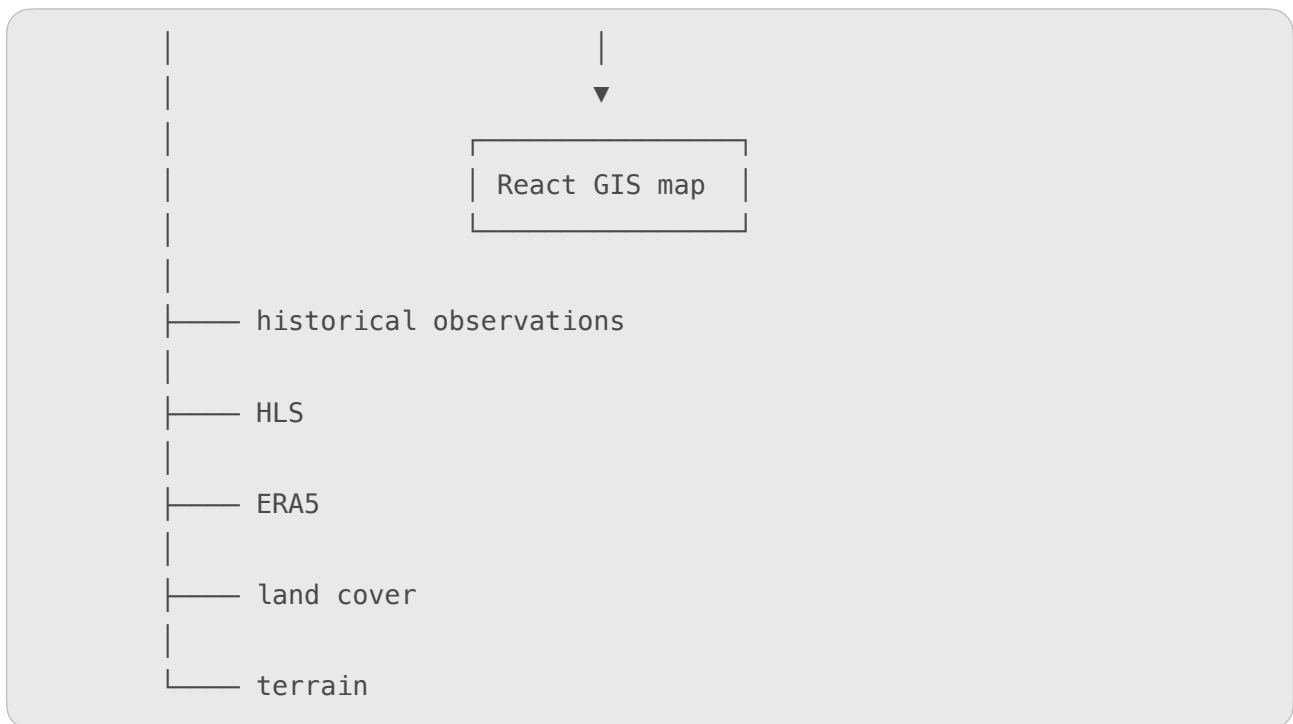
THERMAL SIGNATURE
+
SPATIAL CONTEXT
+
TEMPORAL BEHAVIOR
+
SATELLITE CONTEXT
+
WEATHER
+
TERRAIN
↓
LIGHTGBM
↓
INDUSTRIAL / NON-INDUSTRIAL

It also explains **why simply using FIRMS alone isn't sufficient for your classification problem**.

The most important database relationship

Ultimately, I would structure the relationships approximately like this:





The key idea is that **PostGIS is your spatial/context engine, FastAPI is your orchestration/feature-construction layer, LightGBM is your decision function, PostgreSQL is your persistent state/history, and React is your visualization layer.**

If you want to go even deeper next, the natural next step is to design the **actual PostgreSQL/PostGIS schema**: every table, column, PostgreSQL data type, primary key, foreign key, PostGIS geometry type/SRID, indexes (especially GiST indexes), and exactly which SQL/PostGIS query generates each of your ~40 features. That would give you the database blueprint you can directly implement.

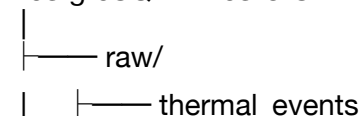
Absolutely. Let's turn your architecture into an implementable **PostgreSQL + PostGIS schema and feature-generation pipeline**.

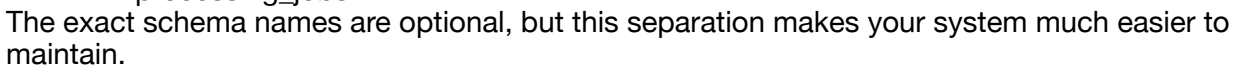
One correction from the previous answer first: your listed vector contains **48 features if every listed item is treated as one model column**. `land_cover_class`, `day_night`, and potentially confidence/sensor-related fields need an explicit encoding decision before training. I would preserve the original categorical values in the database and create a deterministic numerical representation for LightGBM.

1. Database architecture

I recommend separating the database into four logical layers:

PostgreSQL + PostGIS





because distances naturally come back in meters.

```
frp DOUBLE PRECISION,  
brightness temperature DOUBLE PRECISION,
```

confidence DOUBLE PRECISION,
day_night CHAR(1),

created_at TIMESTAMPTZ DEFAULT now()

);

Create the spatial index:

```
CREATE INDEX idx_thermal_events_location  
ON raw.thermal_events  
USING GIST(location);
```

And a time index:

```
CREATE INDEX idx_thermal_events_time  
ON raw.thermal_events(acquisition_time);
```

And probably a combined query-supporting index:

```
CREATE INDEX idx_thermal_events_sensor_time  
ON raw.thermal_events(sensor, acquisition_time);
```

4. Why these columns exist

For example:

event_id

is your internal identifier.

source_event_id

lets you trace the record back to FIRMS/source data.

acquisition_time

is extremely important because your model isn't just spatial.

You need to know:

Where was the anomaly?

and:

When did it occur?

The combination is effectively:

(event location, event time)

rather than merely:

(latitude, longitude)

5. Industrial facilities

Create:

```
CREATE TABLE reference.industrial_facilities (  
    facility_id BIGSERIAL PRIMARY KEY,
```

facility_name TEXT,

facility_type TEXT NOT NULL,

source TEXT,

source_id TEXT,

latitude DOUBLE PRECISION,

longitude DOUBLE PRECISION,

location GEOGRAPHY(Point, 4326),

facility_geometry GEOMETRY(Geometry, 4326),

active BOOLEAN DEFAULT TRUE,

created_at TIMESTAMPTZ DEFAULT now()

);

Spatial indexes:

```
CREATE INDEX idx_industrial_facilities_location
```

```
ON reference.industrial_facilities
USING GIST(location);
```

```
CREATE INDEX idx_industrial_facilities_geometry
ON reference.industrial_facilities
USING GIST(facility_geometry);
```

6. Facility type should be standardized

Don't allow random strings such as:

"Power"

"Power Plant"

"Thermal Power Station"

"TPP"

to represent the same category.

Use a controlled vocabulary.

For example:

REFINERY

OIL_GAS

POWER_PLANT

STEEL

CEMENT

STEEL_CEMENT

MINE

LANDFILL

LNG

OTHER_INDUSTRIAL

You can create a lookup table:

```
CREATE TABLE reference.facility_types (
  facility_type_code TEXT PRIMARY KEY,
  description TEXT
);
```

Then:

industrial_facilities.facility_type
can reference it.

7. Land-cover table

If your land-cover source is vector-based:

```
CREATE TABLE reference.land_cover (
  land_cover_id BIGSERIAL PRIMARY KEY,

  class_code INTEGER NOT NULL,
  class_name TEXT NOT NULL,

  geometry GEOMETRY(MultiPolygon, 4326),

  source TEXT,
  resolution_m INTEGER,

  valid_from DATE,
  valid_to DATE
);
```

Spatial index:

```
CREATE INDEX idx_land_cover_geometry
ON reference.land_cover
USING GIST(geometry);
```

If your land-cover source is raster, I would **not automatically convert a huge raster into millions of PostgreSQL polygons**.

For large raster datasets, keep the raster/object externally and store metadata/reference information in PostgreSQL.

8. HLS imagery metadata

You need to know which image produced:

ndvi_pre

nbr_pre

ndbi_pre

So create something like:

```
CREATE TABLE raw.satellite_images (  
  image_id BIGSERIAL PRIMARY KEY,  
  
  source TEXT NOT NULL,  
  collection TEXT,  
  
  acquisition_time TIMESTAMPTZ NOT NULL,  
  
  footprint GEOMETRY(Polygon, 4326),  
  
  cloud_cover DOUBLE PRECISION,  
  
  resolution_m DOUBLE PRECISION,  
  
  processing_level TEXT,  
  
  asset_reference TEXT,  
  
  created_at TIMESTAMPTZ DEFAULT now()  
);
```

Spatial index:

```
CREATE INDEX idx_satellite_images_footprint  
ON raw.satellite_images  
USING GIST(footprint);
```

```
CREATE INDEX idx_satellite_images_time  
ON raw.satellite_images(acquisition_time);
```

9. Weather

Your weather data is fundamentally spatiotemporal.

A simplified representation:

```
CREATE TABLE raw.weather_observations (  
  weather_id BIGSERIAL PRIMARY KEY,  
  
  observation_time TIMESTAMPTZ NOT NULL,  
  
  grid_id TEXT,  
  
  location GEOGRAPHY(Point, 4326),  
  
  wind_speed DOUBLE PRECISION,  
  
  wind_direction DOUBLE PRECISION,  
  
  vpd DOUBLE PRECISION,  
  
  rainfall DOUBLE PRECISION,  
  
  source TEXT  
);
```

Index:

```
CREATE INDEX idx_weather_location  
ON raw.weather_observations  
USING GIST(location);
```

```
CREATE INDEX idx_weather_time
ON raw.weather_observations(observation_time);
```

For actual ERA5-scale datasets, however, I would probably keep the full gridded data in an optimized scientific-data store/object storage and retain the extracted values used for each event in your ML feature table.

10. Terrain

You don't necessarily need a giant terrain table.

For the ML system, you can store the extracted values associated with the event:

elevation

slope

But if you want a terrain reference table:

```
CREATE TABLE reference.terrain (
  terrain_id BIGSERIAL PRIMARY KEY,

  location GEOGRAPHY(Point, 4326),

  elevation DOUBLE PRECISION,
  slope DOUBLE PRECISION,

  source TEXT,
  resolution_m DOUBLE PRECISION
);
```

For a raster DEM, again, don't unnecessarily explode the raster into point rows.

11. The most important table: feature_vectors

This should contain the **exact values supplied to LightGBM**.

I would structure it like this:

```
CREATE TABLE ml.feature_vectors (
  feature_vector_id BIGSERIAL PRIMARY KEY,

  event_id BIGINT NOT NULL,

  feature_version TEXT NOT NULL,

  created_at TIMESTAMPTZ DEFAULT now(),

  -- FIRMS / thermal
  frp_log1p DOUBLE PRECISION,
  frp_density DOUBLE PRECISION,
  bt_fire DOUBLE PRECISION,
  bt_delta DOUBLE PRECISION,
  confidence DOUBLE PRECISION,
  day_night DOUBLE PRECISION,
  sensor_agreement DOUBLE PRECISION,

  -- Morphology
  cluster_area DOUBLE PRECISION,
  cluster_compactness DOUBLE PRECISION,
  centroid_shift DOUBLE PRECISION,

  -- Temporal
  historical_frp_mean DOUBLE PRECISION,
  historical_frp_std DOUBLE PRECISION,
  robust_frp_zscore DOUBLE PRECISION,
  detection_frequency_30d DOUBLE PRECISION,
  detection_frequency_90d DOUBLE PRECISION,
  persistence_90d DOUBLE PRECISION,
  active_year_fraction DOUBLE PRECISION,
```

```

active_month_fraction DOUBLE PRECISION,
night_detection_fraction DOUBLE PRECISION,
days_since_previous_detection DOUBLE PRECISION,
time_since_first_detection DOUBLE PRECISION,

-- Proximity
distance_to_refinery DOUBLE PRECISION,
distance_to_oil_gas_facility DOUBLE PRECISION,
distance_to_power_plant DOUBLE PRECISION,
distance_to_steel_cement_plant DOUBLE PRECISION,
distance_to_mine DOUBLE PRECISION,
distance_to_landfill DOUBLE PRECISION,
distance_to_nearest_industrial_structure DOUBLE PRECISION,
in_industrial_polygon DOUBLE PRECISION,
industrial_facility_count_3km DOUBLE PRECISION,

-- Surroundings
land_cover_class DOUBLE PRECISION,
cropland_fraction_1km DOUBLE PRECISION,
forest_fraction_1km DOUBLE PRECISION,
industrial_fraction_1km DOUBLE PRECISION,
bare_soil_fraction_1km DOUBLE PRECISION,

-- HLS
ndvi_pre DOUBLE PRECISION,
nbr_pre DOUBLE PRECISION,
ndbi_pre DOUBLE PRECISION,
builtup_fraction_pre DOUBLE PRECISION,
optical_valid_fraction DOUBLE PRECISION,
hls_days_before_event DOUBLE PRECISION,
data_availability_flag DOUBLE PRECISION,

-- Weather
wind_speed DOUBLE PRECISION,
wind_direction_sin DOUBLE PRECISION,
wind_direction_cos DOUBLE PRECISION,
vpd DOUBLE PRECISION,
rainfall_72h DOUBLE PRECISION,

-- Terrain
elevation DOUBLE PRECISION,
slope DOUBLE PRECISION,

```

```

FOREIGN KEY(event_id)
REFERENCES raw.thermal_events(event_id)

```

```

);
Add:
CREATE INDEX idx_feature_vectors_event
ON ml.feature_vectors(event_id);

```

12. But don't make the database your only feature representation

For inference, your Python code should construct something like:

```

X = [
    frp_log1p,
    frp_density,
    bt_fire,
    ...
    slope
]

```

Then:

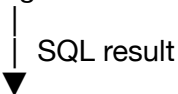
```
prediction = model.predict_proba(X)
```

The database is your **persistent record**.

The in-memory NumPy array is the **actual computational representation**.

Think:

PostgreSQL



Python dictionary/object



NumPy array



LightGBM

13. Feature schema table

I strongly recommend one more table.

```
CREATE TABLE ml.feature_schema (
  feature_version TEXT NOT NULL,
  feature_index INTEGER NOT NULL,
  feature_name TEXT NOT NULL,
  data_type TEXT NOT NULL,
  source TEXT,
  transformation TEXT,
  required BOOLEAN DEFAULT TRUE,

  PRIMARY KEY(feature_version, feature_index)
);
```

Then your model has a formal contract.

For example:

v1

```
0 frp_log1p
1 frp_density
2 bt_fire
3 bt_delta
4 confidence
```

...

```
47 slope
```

This protects you from accidentally changing the order.

14. Categorical features need special attention

You have:

land_cover_class

Suppose:

forest

is stored as:

"FOREST"

in your database.

You shouldn't randomly convert it to:

FOREST = 1

CROPLAND = 2

INDUSTRIAL = 3

unless your training pipeline deliberately uses that encoding.

Otherwise you're introducing artificial numerical ordering.

You need to decide on a consistent LightGBM encoding strategy.

For example, depending on your LightGBM setup, you can use categorical features explicitly, or encode them deterministically.

The critical requirement is:

training encoding == inference encoding

15. Prediction table

```
CREATE TABLE ml.predictions (  
  prediction_id BIGSERIAL PRIMARY KEY,  
  
  event_id BIGINT NOT NULL,  
  
  feature_vector_id BIGINT NOT NULL,  
  
  model_id BIGINT NOT NULL,  
  
  predicted_class TEXT NOT NULL,  
  
  industrial_probability DOUBLE PRECISION,  
  
  non_industrial_probability DOUBLE PRECISION,  
  
  decision_threshold DOUBLE PRECISION,  
  
  prediction_time TIMESTAMPTZ DEFAULT now(),  
  
  geometry GEOGRAPHY(Point, 4326),  
  
  FOREIGN KEY(event_id)  
    REFERENCES raw.thermal_events(event_id),  
  
  FOREIGN KEY(feature_vector_id)  
    REFERENCES ml.feature_vectors(feature_vector_id),  
  
  FOREIGN KEY(model_id)  
    REFERENCES ml.model_registry(model_id)  
);  
Spatial index:  
CREATE INDEX idx_predictions_geometry  
ON ml.predictions  
USING GIST(geometry);
```

16. Model registry

```
CREATE TABLE ml.model_registry (  
  model_id BIGSERIAL PRIMARY KEY,  
  
  model_name TEXT NOT NULL,  
  model_version TEXT NOT NULL,  
  
  feature_version TEXT NOT NULL,  
  
  model_type TEXT NOT NULL,  
  
  model_location TEXT NOT NULL,  
  
  model_hash TEXT,  
  
  training_dataset_version TEXT,  
  
  training_timestamp TIMESTAMPTZ,  
  
  decision_threshold DOUBLE PRECISION,  
  
  metrics JSONB,
```

```
active BOOLEAN DEFAULT FALSE,  
created_at TIMESTAMPTZ DEFAULT now()  
);
```

Example:

```
model_name:  
industrial_fire_classifier
```

```
model_version:  
lgbm_v3
```

```
feature_version:  
features_v5
```

```
model_type:  
LightGBM
```

```
decision_threshold:  
0.50
```

17. Why model hash matters

Suppose you have:

```
lgbm_v3
```

and someone accidentally replaces the model file.

Your database still says:

```
lgbm_v3
```

but the actual binary is different.

A hash lets you detect this.

Conceptually:

```
model file
```

↓

```
SHA-256
```

↓

```
ABC123....
```

Store that hash in:

```
model_hash
```

Now you can prove which model artifact generated a prediction.

18. Now let's implement the actual inference workflow

Your endpoint could conceptually be:

```
POST /api/v1/predict
```

Request:

```
{  
  "latitude": 19.8765,  
  "longitude": 73.4567  
}
```

FastAPI receives it.

19. FastAPI → PostGIS

First:

```
lat/lon
```

↓

```
validate
```

↓

```
POINT(lon lat)
```

Then find the relevant thermal event.

Conceptually:

```
SELECT *  
FROM raw.thermal_events
```

```

WHERE ST_DWithin(
    location,
    ST_SetSRID(
        ST_MakePoint(:longitude, :latitude),
        4326
    )::geography,
    :radius_meters
)
ORDER BY ST_Distance(
    location,
    ST_SetSRID(
        ST_MakePoint(:longitude, :latitude),
        4326
    )::geography
)
LIMIT 1;

```

The actual radius/selection policy should be explicitly defined by your application.

20. Thermal feature extraction

You then retrieve:

FRP

BT

confidence

sensor

day/night

and calculate:

frp_log1p

frp_density

bt_fire

bt_delta

...

Important:

Don't calculate the training features one way and inference features another way.

The best architecture is to put feature-generation logic into reusable Python functions:

get_thermal_features()

get_morphology_features()

get_temporal_features()

get_proximity_features()

get_surrounding_features()

get_imagery_features()

get_weather_features()

get_terrain_features()

Then:

build_feature_vector()

calls all of them.

21. Proximity query

For:

distance_to_refinery

you can query something like:

```
SELECT
```

```
    ST_Distance(
        :event_point,
        location
    ) AS distance_m
```

```
FROM reference.industrial_facilities
```

```
WHERE facility_type = 'REFINERY'
```

```
ORDER BY :event_point <-> location
```

```
LIMIT 1;
```

The important piece is:

<->

which can help PostGIS perform nearest-neighbor spatial searches with the appropriate GiST index.

You repeat the concept for:

OIL_GAS

POWER_PLANT

STEEL

CEMENT

MINE

LANDFILL

22. industrial_facility_count_3km

This one becomes:

```
SELECT COUNT(*)
FROM reference.industrial_facilities
WHERE ST_DWithin(
    location,
    :event_point,
    3000
);
```

Result:

5

So:

industrial_facility_count_3km = 5

23. in_industrial_polygon

If you have industrial polygons:

```
SELECT EXISTS (
    SELECT 1
    FROM reference.industrial_facilities
    WHERE ST_Intersects(
        facility_geometry,
        :event_geometry
    )
);
```

Result:

TRUE

Convert according to your model schema:

TRUE → 1

FALSE → 0

24. Surrounding fractions

Create a 1 km buffer:

event

↓

1 km buffer

↓

intersect land cover

Conceptually:

ST_Buffer(event_geometry::geography, 1000)

Then calculate area intersections.

For example:

buffer area

	forest = 18%
	crop = 7%
	industrial = 62%
	bare soil = 13%

Those become your four fraction features.

25. Temporal feature generation

This deserves special attention.

For a new event:

T = 2026-09-21 13:42

you query:

T - 30 days → T

for:

detection_frequency_30d

and:

T - 90 days → T

for:

detection_frequency_90d

Then:

historical_frp_mean

historical_frp_std

come from the historical FRP observations.

The key is that **the current event should not accidentally leak into historical statistics.**

For training, this is especially important.

26. Avoiding data leakage

Suppose you're training a model using an event from:

September 21

You cannot calculate its historical features using detections from:

September 22

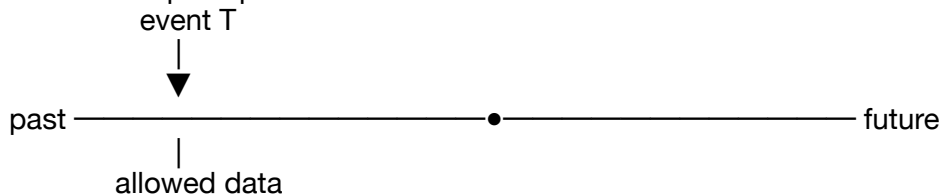
September 25

October 1

because those events had not happened yet.

Otherwise your training model gets information from the future.

The correct principle is:



For historical features:

use data < T

not:

use data around T

This is one of the most important things to get right in your ML pipeline.

27. HLS feature generation

Your imagery pipeline should have a deterministic rule such as:

Find valid HLS image

before event

within maximum allowed lookback

For example:

event = September 21

candidate images:

September 20 → cloudy

September 18 → invalid

September 16 → valid

Then:

selected image = September 16

and:

hls_days_before_event = 5

Your exact lookback policy should be defined before training.

28. Weather feature generation

Similarly:

event



ERA5 grid cell



matching time/window



weather values

Then:

wind_speed

wind_direction

VPD

rainfall_72h

and:

wind_direction



sin()

cos()



wind_direction_sin

wind_direction_cos

29. Finally create the feature vector

At this point your Python process has something conceptually like:

```
features = {  
    "frp_log1p": ...,  
    "frp_density": ...,  
    "bt_fire": ...,  
    ...  
    "elevation": ...,  
    "slope": ...  
}
```

Then use the feature schema to order it:

feature_schema



[feature 0,
feature 1,
feature 2,
...

feature 47]

Then:

Python list



NumPy array



LightGBM

30. LightGBM inference

Conceptually:

probabilities = model.predict_proba(X)

You might get:

[0.08, 0.92]

meaning:

$P(\text{non-industrial}) = 0.08$

$P(\text{industrial}) = 0.92$

Then:

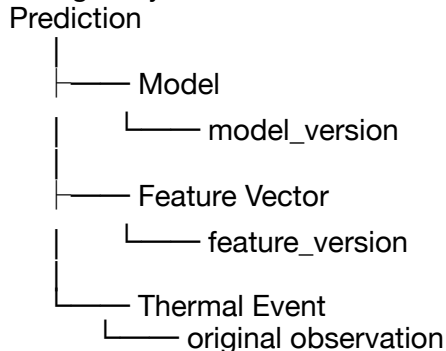
industrial_probability = 0.92
and your configured threshold determines the label.

31. Store the exact result

You should insert:

event_id
feature_vector_id
model_id
prediction
probabilities
threshold
prediction_time
geometry

This gives you an audit chain:



That is a very strong architecture for your project.

32. What React should retrieve

Your React map shouldn't need all the raw data every time.

Create a response/API specifically for visualization.

For example:

```
{
  "prediction_id": 500183,
  "event_id": 182734,

  "latitude": 19.8765,
  "longitude": 73.4567,

  "classification": "INDUSTRIAL_FIRE",

  "industrial_probability": 0.92,

  "event_time": "2026-09-21T13:42:00Z",

  "frp": 42.7,

  "model_version": "lgbm_v3"
}
```

React can then put that point on the map.

33. Your map query can be spatial

This is where PostGIS becomes very useful.

Suppose React displays the map viewport:

bounding box

You don't want to download every prediction in India.

Instead:

React

↓

map bounding box

↓

FastAPI



PostGIS



predictions inside bbox



GeoJSON



React

PostGIS can generate GeoJSON directly:

```
SELECT json_build_object(  
  'type', 'Feature',  
  'geometry', ST_AsGeoJSON(geometry)::json,  
  'properties', json_build_object(  
    'prediction_id', prediction_id,  
    'classification', predicted_class,  
    'probability', industrial_probability  
  )  
)  
FROM ml.predictions  
WHERE geometry && ST_MakeEnvelope(...);  
That is a natural architecture for your GIS frontend.
```

34. I would also add a processing_jobs table

Because your pipeline will eventually have failures.

For example:

FIRMS found



PostGIS successful



HLS unavailable



prediction cannot be generated
You need to know what happened.

Create:

```
CREATE TABLE monitoring.processing_jobs (  
  job_id BIGSERIAL PRIMARY KEY,  
  
  event_id BIGINT,  
  
  status TEXT,  
  
  current_stage TEXT,  
  
  error_message TEXT,  
  
  started_at TIMESTAMPTZ,  
  
  completed_at TIMESTAMPTZ,  
  
  created_at TIMESTAMPTZ DEFAULT now()  
);
```

Possible states:

RECEIVED

FEATURE_EXTRACTION

FEATURES_READY

PREDICTING

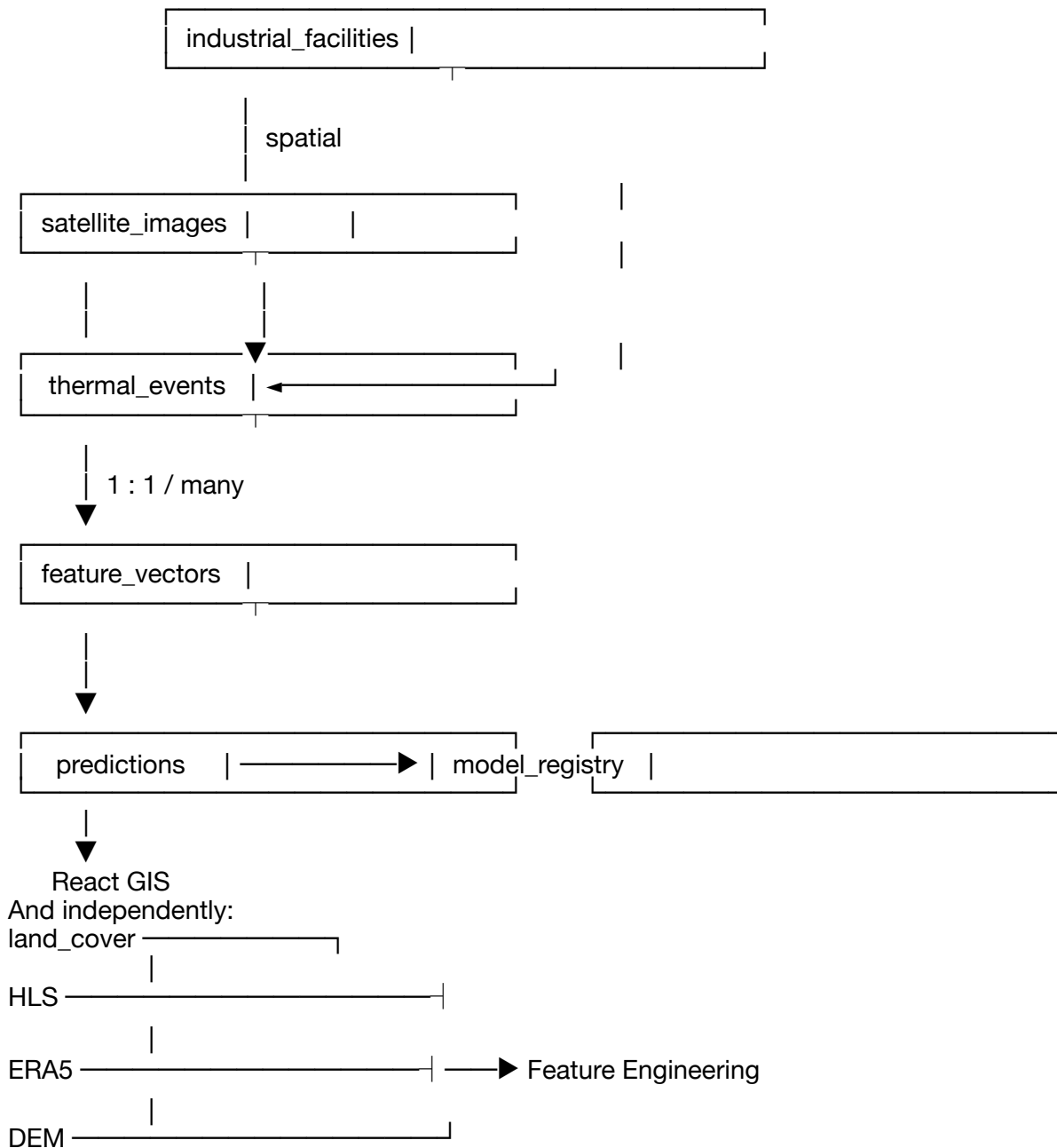
COMPLETED

FAILED

This will save you a lot of debugging time.

35. The complete database relationship

Your final conceptual ER structure becomes:



36. One thing I strongly recommend changing in your architecture

Don't make this:

React



FastAPI



calculate everything



LightGBM

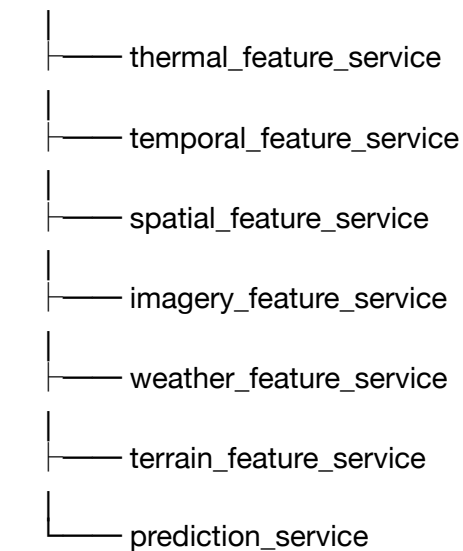
inside one giant API function.

Instead:

FastAPI



event_service



Then:

prediction_service



feature_service



feature vector



LightGBM

This will make testing much easier.

37. The exact responsibility of each technology

Your stack now has a very clear division:

PostgreSQL

Stores:

events
facilities
metadata
features
predictions
models
processing state

PostGIS

Performs:

distance
intersection
containment
buffer
nearest-neighbor
spatial filtering

FastAPI

Performs:

API
validation
orchestration
feature construction
model invocation
database interaction

Python/NumPy

Performs:

mathematical transformations

feature vector assembly
numerical preprocessing

LightGBM

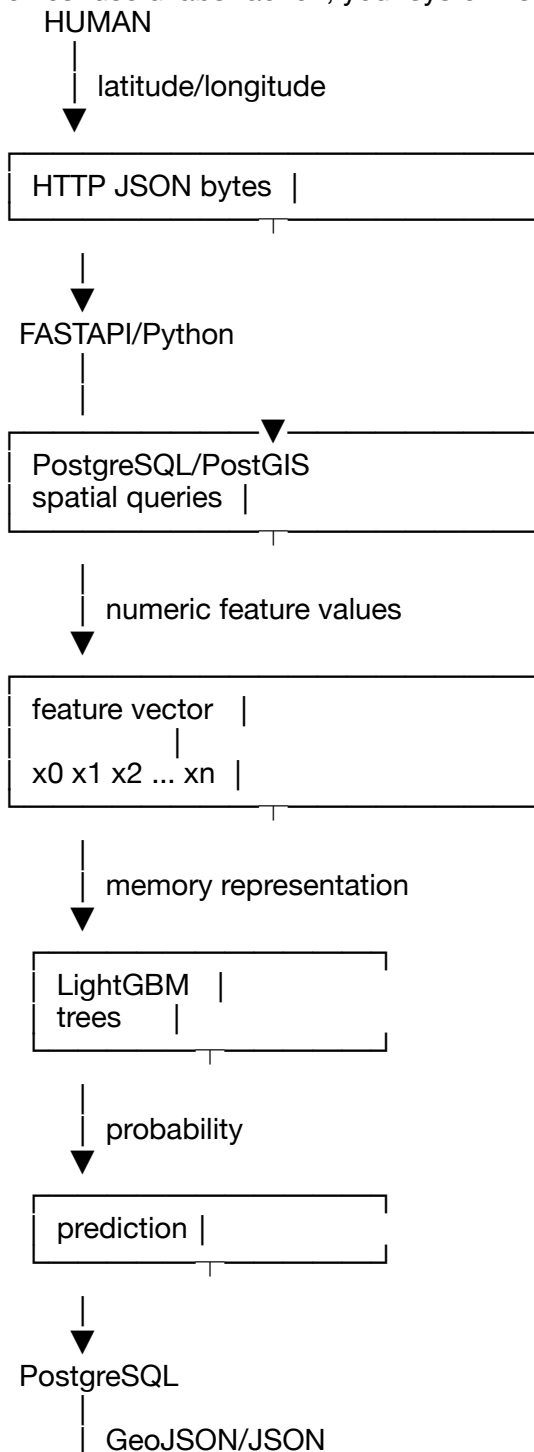
Performs:
classification
probability estimation

React

Performs:
map
user interaction
visualization
prediction display

38. The final binary/data-flow picture

At the lowest useful abstraction, your system is essentially:



▼
React

▼
GIS MAP

The critical insight is:

Latitude/longitude is not the ML input. It is the key that lets your system retrieve and derive the ML input.

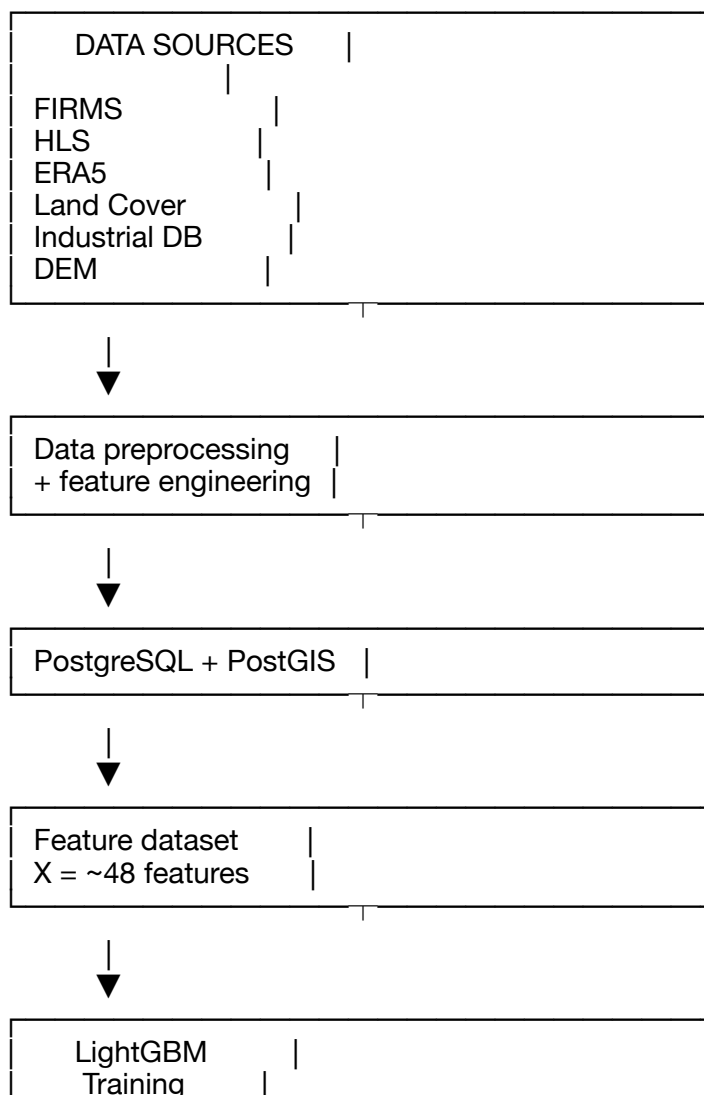
Your actual ML input is the feature vector generated from **thermal + morphology + temporal history + proximity + surroundings + HLS + weather + terrain.**

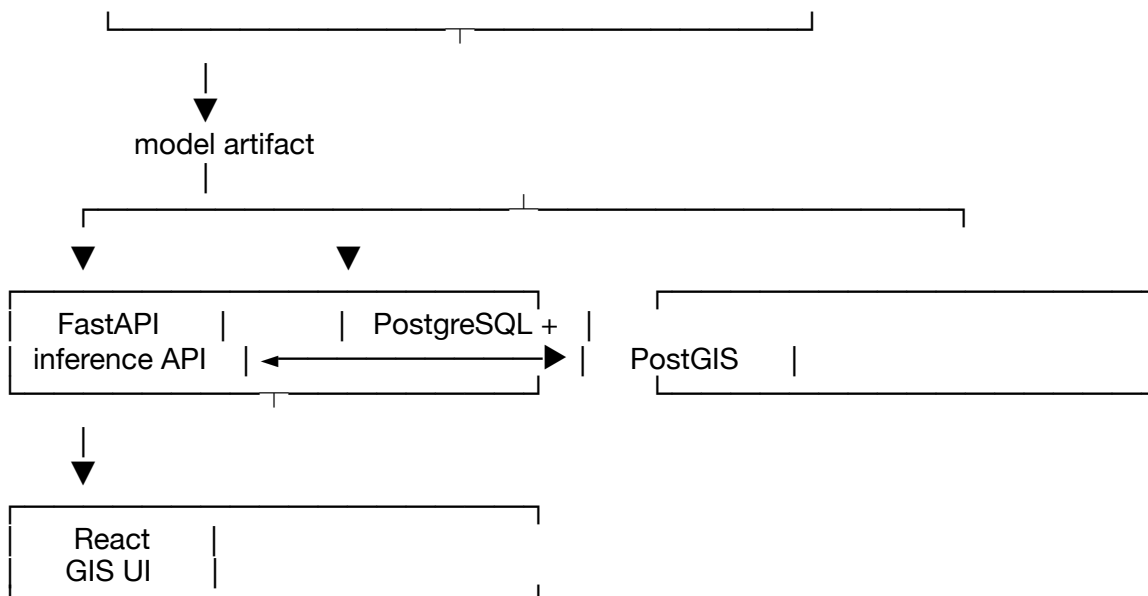
That distinction should be central to both your implementation and your project presentation.

For your specific architecture—**FIRMS + HLS + ERA5 + PostGIS + FastAPI + LightGBM + React**—the hardware requirements are actually quite manageable **if you are training LightGBM on tabular feature vectors rather than training a deep-learning model directly on satellite images.**

The expensive part is likely to be **geospatial data storage and preprocessing**, not the LightGBM model itself.

1. Recommended overall architecture





2. Hardware requirements for development/training

Minimum viable machine

For a student/research prototype:

Component	Minimum
CPU	4 cores
RAM	16 GB
Storage	512 GB SSD
GPU	Not required
Internet	Stable broadband
OS	Ubuntu 22.04/24.04, Windows 11, or macOS

This can train a LightGBM classifier if your feature dataset is reasonably sized.

For example, if your training data looks like:

500,000 events

x

48 features

you're still dealing with a relatively small tabular ML dataset compared with image/deep-learning workloads.

3. Recommended training machine

For your project, I'd target:

Component	Recommendation
CPU	8–16 cores
RAM	32 GB
Storage	1–2 TB NVMe SSD
GPU	Optional
Network	100 Mbps+
OS	Ubuntu 24.04 LTS

Something around:

8-core/16-thread CPU

32 GB RAM

1 TB NVMe
would be a very reasonable development machine.
You **do not need an expensive NVIDIA GPU just for LightGBM.**

4. Why CPU matters more than GPU here

Your model is:
LightGBM
and your input is:
tabular vector
rather than:
satellite image → CNN/Transformer
So your computational workload is roughly:
48 numbers

↓
decision trees
↓
decision trees
↓
prediction
rather than:
10,000 × 10,000 satellite image
↓
millions of convolution operations
↓
deep neural network
Therefore:
CPU: VERY IMPORTANT
RAM: IMPORTANT
SSD: IMPORTANT
GPU: OPTIONAL

5. But there is a catch: your geospatial preprocessing

Your LightGBM training isn't likely to be your biggest hardware problem.

This could be:

FIRMS
+
HLS
+
ERA5
+
land cover
+
industrial polygons
+
DEM

For example, imagine you have:

10 million FIRMS observations

and need to calculate:

distance to industrial facilities

for every observation.

That's where computational cost starts increasing.

Your pipeline becomes:

10M events
↓
spatial queries
↓
historical queries
↓
HLS extraction

↓
 weather extraction
 ↓
 land-cover intersection
 ↓
 feature dataset
 This is much heavier than:
 $10M \times 48$
 ↓
 LightGBM

6. Recommended RAM based on dataset size

A useful rough guide:

Dataset	RAM
< 500k events	16 GB
500k–2M	32 GB
2M–10M	64 GB
>10M	64–128 GB or distributed processing

These aren't strict requirements; efficient chunking can let you work with less RAM.

You should **not load every raw satellite product into RAM simultaneously**.

Instead:

disk/object storage

↓
 small chunk
 ↓
 process
 ↓
 write result
 ↓
 next chunk

7. Storage requirements

This is where I would give your project more attention.

You potentially have:

FIRMS data
 HLS metadata
 HLS imagery
 ERA5 data
 land-cover data
 DEM
 industrial databases
 PostgreSQL
 PostGIS indexes
 feature datasets
 model artifacts
 logs

So I would recommend:

Development

1 TB NVMe SSD

Serious regional deployment

2–4 TB SSD

Large national/global deployment

Don't rely on one local disk.

Use:

object storage

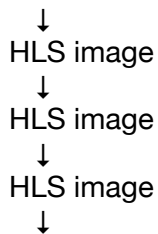
+
PostgreSQL/PostGIS

8. Don't store all satellite imagery inside PostgreSQL

This is an important architecture decision.

Don't do:

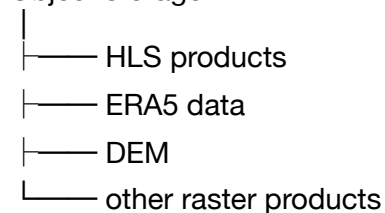
PostgreSQL



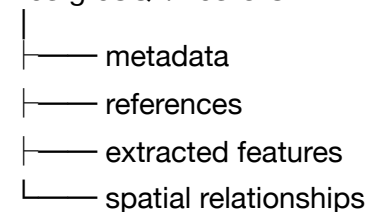
...
for a huge dataset.

Instead:

Object storage



PostgreSQL/PostGIS



So PostgreSQL knows:

image_id

source

acquisition_time

footprint

asset_reference

and your actual large raster stays in appropriate storage.

9. Training software requirements

Your training environment could be:

Ubuntu 24.04

Python 3.11/3.12

Then:

numpy

pandas

scikit-learn

lightgbm

geopandas

shapely

pyproj

rasterio

xarray

rioxarray

SQLAlchemy

psycopg

Potentially also:

dask

if your geospatial preprocessing becomes large.

10. Database software

You need:

PostgreSQL

+

PostGIS

For example:

PostgreSQL 16/17

PostGIS compatible with your PostgreSQL version

You should keep the database version consistent between development and production where practical.

11. Backend software

Your backend:

Python

FastAPI

Uvicorn

Potential dependencies:

FastAPI

Uvicorn

Pydantic

SQLAlchemy

psycopg

GeoAlchemy2

LightGBM

NumPy

Pandas

Shapely

PyProj

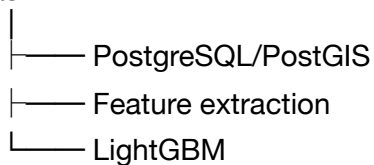
Architecture:

React

↓
HTTP/JSON



FastAPI



12. Frontend software

Your React application needs:

Node.js

npm/pnpm

React

For GIS visualization, you can use a mapping library such as:

MapLibre GL JS

or:

Leaflet

or another WebGL/GIS mapping framework.

Your frontend receives GeoJSON/API data from FastAPI.

For example:

FastAPI



GeoJSON



React

↓
map layer

13. Development machine I'd recommend for you

If you're building the complete project yourself, I'd target:

CPU:

8-core / 16-thread

RAM:

32 GB

Storage:

1 TB NVMe SSD

GPU:

Not required

Network:

Good broadband

OS:

Ubuntu 24.04 LTS

That's a very comfortable setup for:

PostgreSQL

PostGIS

Python

LightGBM

FastAPI

React

GIS processing

running locally.

14. If you already have a laptop

You don't necessarily need to buy anything.

If you have:

16 GB RAM

modern 4–8 core CPU

512 GB SSD

you can absolutely start development and model training.

The first bottleneck you'll encounter is more likely to be:

data acquisition

↓
data preprocessing

↓
storage
rather than:
LightGBM training

15. GPU: when would you actually need one?

For your current architecture:

LightGBM

GPU is optional.

You would start needing a substantial GPU if you change the project to something like:

HLS image

↓
CNN

↓
classification
or:

HLS temporal sequence



Vision Transformer



classification

Then you might look at:

NVIDIA GPU

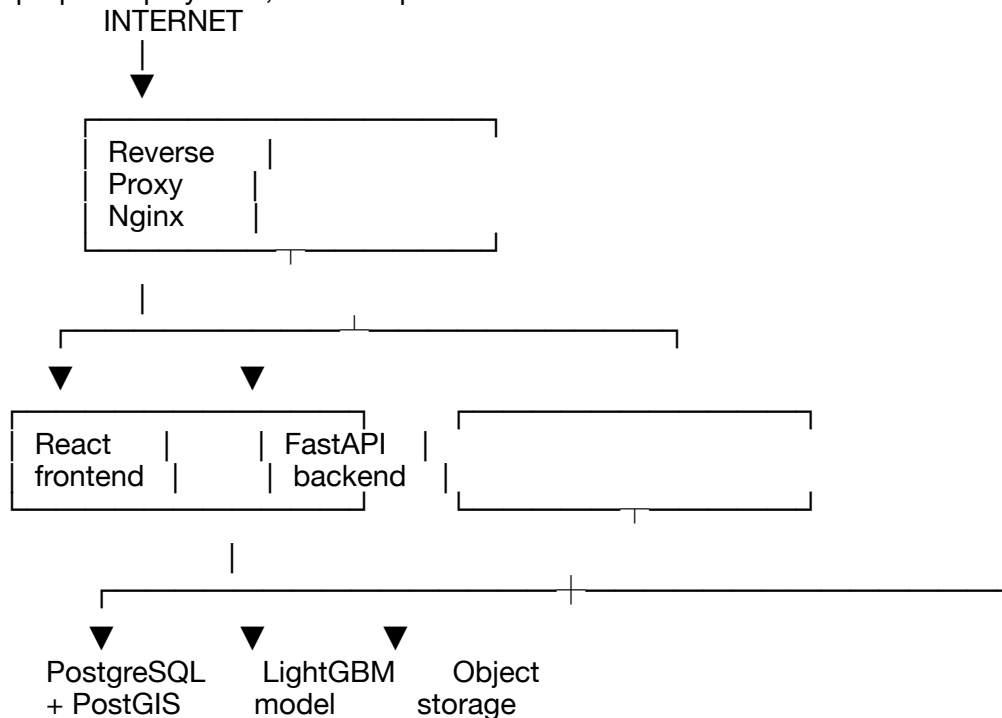
8–24 GB VRAM

depending on the model.

But **don't add GPU hardware simply because it's an AI project.**

16. Production deployment

For a proper deployment, I would split the services.



17. Production server requirements

For a moderate deployment:

CPU: 8 vCPU

RAM: 32 GB

SSD: 500 GB–1 TB

GPU: None

would be a reasonable starting point.

If you're only serving predictions from already-computed features:

4 vCPU

16 GB RAM

could potentially be enough.

But your actual requirements depend heavily on concurrent users and how much feature extraction happens synchronously during a prediction request.

18. Don't make the API download everything synchronously

This is probably the biggest deployment architecture issue to think about.

Bad:

User clicks location



FastAPI



download HLS



download ERA5



process imagery



run spatial queries



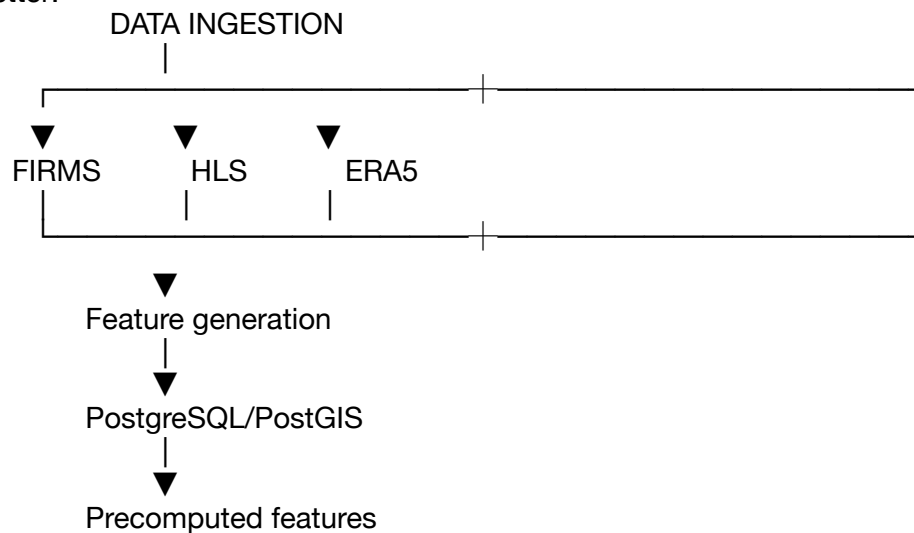
LightGBM



response

The user could end up waiting a long time.

Better:



Then:

User



lat/lon



FastAPI



PostGIS



retrieve existing feature vector



LightGBM



response

This is **much faster**.

19. Your system should therefore have two pipelines

This is a major architectural improvement.

Pipeline A — Data/feature pipeline

FIRMS

HLS

ERA5

Land Cover

Industrial DB

DEM



Preprocessing



Feature Engineering



PostGIS/PostgreSQL

This happens periodically.

For example:

hourly

daily

or whenever new satellite data arrives

depending on your data sources and monitoring requirements.

Pipeline B — Prediction/API pipeline

React



lat/lon / event_id



FastAPI



PostGIS



feature vector



LightGBM



prediction



PostGIS



React

This is fast because the expensive data processing already happened.

20. Training pipeline

Your training machine does:

RAW DATA



Data preprocessing



Label generation



Feature engineering



Dataset X, y



Train/validation/test



LightGBM



Evaluation metrics



model artifact



model registry

Your trained model might be something like:

industrial_fire_lgbm_v1.txt

or another LightGBM-supported serialized representation.

21. Training doesn't need to happen on the production server

I would separate:

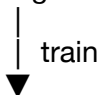
TRAINING

from:

INFERENCE

For example:

Training machine



LightGBM model



Production server



FastAPI

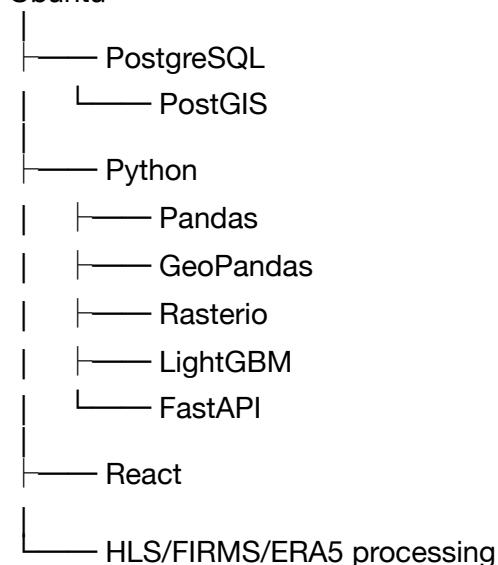
The production server doesn't need all the training infrastructure.

22. A very practical setup for your project

If this is a college/research project, I would build it initially on:

LOCAL MACHINE

Ubuntu

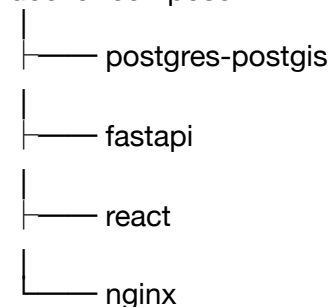


Then Dockerize it.

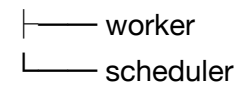
23. Docker architecture

Your docker-compose setup could eventually look like:

docker-compose



And potentially:



when your asynchronous processing becomes necessary.

For example:

worker



HLS extraction



feature generation



PostGIS

while:

FastAPI



prediction API

stays responsive.

24. Production hardware by scale

Prototype

1 machine

8 CPU

32 GB RAM

1 TB SSD

No GPU

Runs:

PostgreSQL

PostGIS

FastAPI

React

LightGBM

Perfectly reasonable for development/demo.

Small deployment

API server

4–8 vCPU

16–32 GB RAM

Database server

8 vCPU

32–64 GB RAM

1–2 TB SSD

Larger deployment

Separate:

Load balancer



FastAPI server 1



FastAPI server 2



FastAPI server 3

Database



PostgreSQL + PostGIS

Object storage



HLS



ERA5

DEM

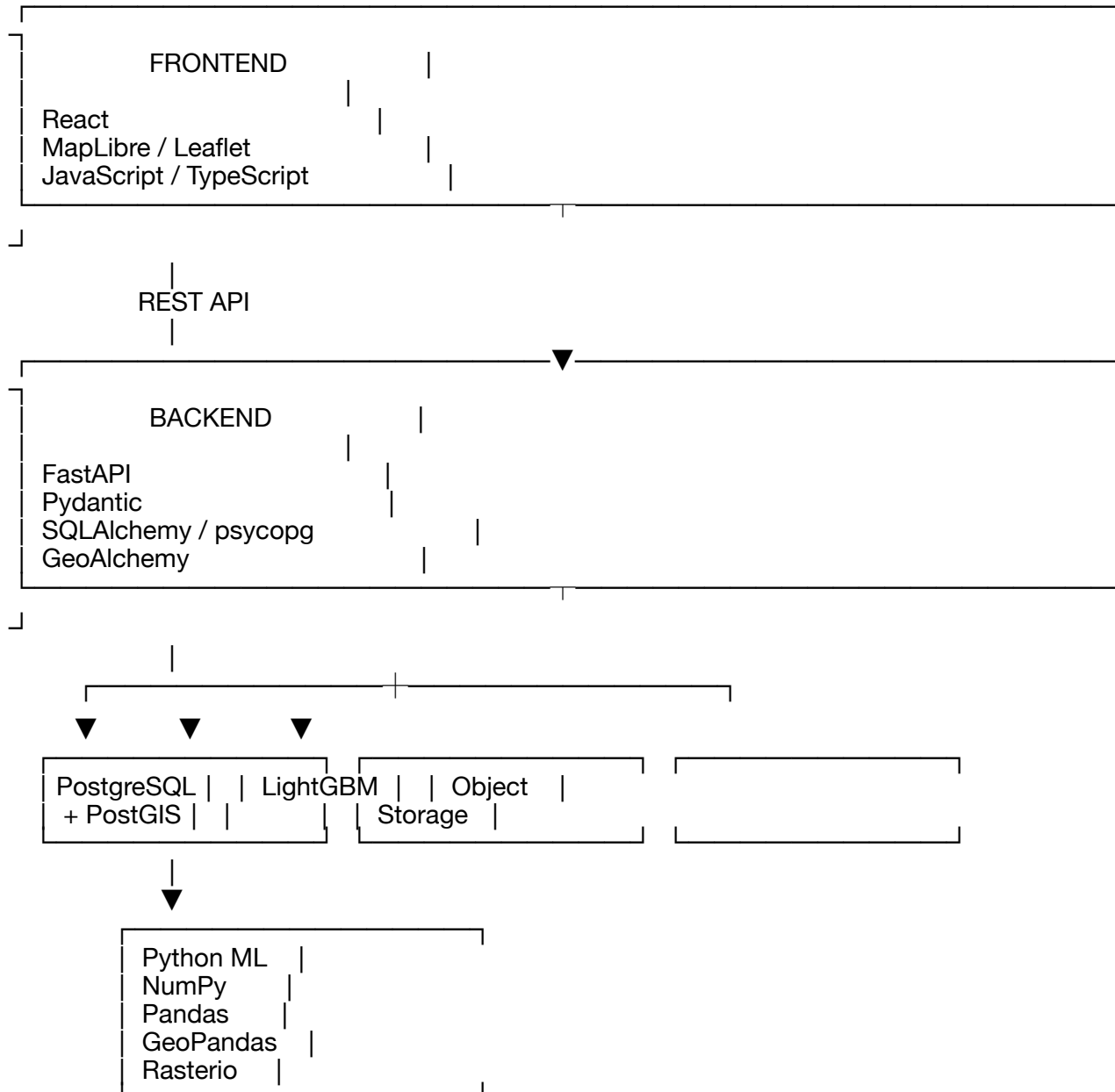
Workers

feature generation

You don't need this complexity for your first version.

25. Software stack I'd recommend

Your final stack can be:



26. One more thing: testing hardware

You should also have enough resources to run:

PostGIS

+

FastAPI

+

React

+

training

simultaneously.

That's why I'd prefer:

32 GB RAM

over spending money on a GPU.

For example:

PostgreSQL/PostGIS → 6–10 GB

Python processing → 4–10 GB

Browser/React → 2–4 GB

OS → 3–5 GB

LightGBM → variable

You can see why 32 GB gives you comfortable headroom.

27. Recommended requirements for *your* project

If you need to put a concise specification in your project report, I'd use something like this:

Hardware — Training/Development

CPU : 8-core processor or higher

RAM : 32 GB

Storage : 1 TB NVMe SSD

GPU : Not required for LightGBM

Network : Stable broadband connection

OS : Ubuntu 22.04/24.04 LTS

Hardware — Deployment

CPU : 4–8 vCPU

RAM : 16–32 GB

Storage : 500 GB–1 TB SSD

GPU : Not required

Network : Reliable high-speed connection

Software

OS

└── Ubuntu Linux

Database

└── PostgreSQL

└── PostGIS

Backend

└── Python

└── FastAPI

└── Uvicorn

└── Pydantic

└── SQLAlchemy

└── GeoAlchemy2

ML

└── LightGBM

└── NumPy

└── Pandas

└── Scikit-learn

└── GeoPandas

└── Shapely

└── Rasterio

└── Xarray/Rioxarray

Frontend

- └── Node.js
- └── React
- └── GIS mapping library

Deployment

- └── Docker
- └── Docker Compose
- └── Nginx

The key takeaway

For **your current LightGBM architecture**, don't design the hardware around GPU computing. Design it around **geospatial preprocessing, RAM, SSD I/O, and PostGIS spatial queries**.

A **32-GB RAM + 8-core CPU + 1-TB NVMe** machine is a very sensible target for development and training. The production system can start around **4–8 vCPU + 16–32 GB RAM**, with the database and heavy feature-generation workloads separated as the system grows.

The next thing I'd design before you write code is the **actual training dataset pipeline**: exactly how you obtain positive/negative labels, how you generate one training row from one thermal event, how you prevent spatial/temporal leakage, how the 48 features are calculated, and how train/validation/test data should be split. **That part is more critical to the validity of your LightGBM model than the hardware.**

Yes. If you don't have a 32 GB RAM machine, you can absolutely use **multiple machines**, but there is an important distinction:

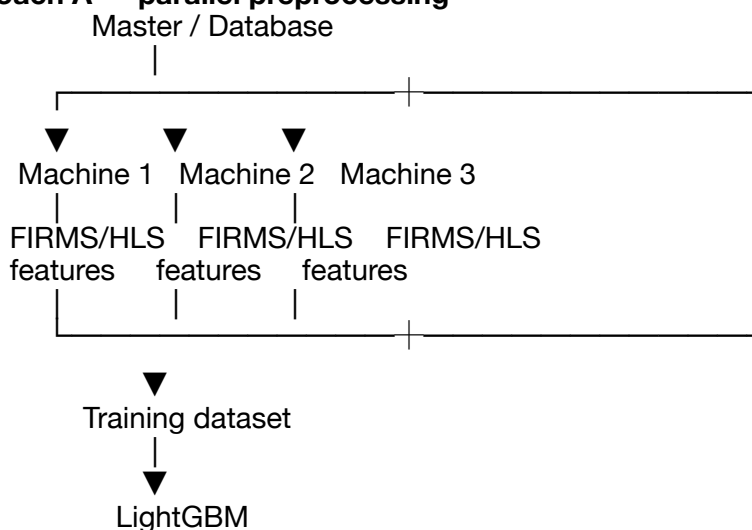
You usually don't want several computers independently training separate LightGBM models and then simply picking one. You want either **distributed LightGBM training** or **distributed preprocessing**, depending on where the bottleneck is.

For your project, I would actually distribute the **data/feature-generation stage first**, because your HLS/ERA5/PostGIS processing is likely to consume more resources than LightGBM training.

1. Two ways to use multiple machines

You can divide the workload like this:

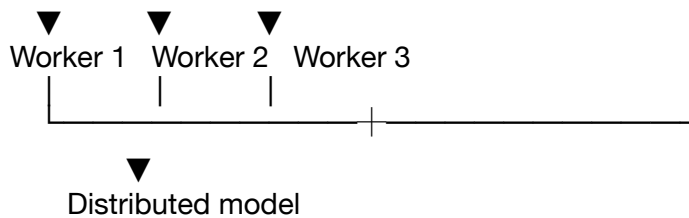
Approach A — parallel preprocessing



This is the approach I'd recommend initially.

Approach B — distributed LightGBM training





LightGBM supports distributed training, so the actual tree-building process can be distributed across machines.

2. You don't necessarily need 32 GB on every machine

Suppose you have:

Machine A → 8 GB RAM

Machine B → 8 GB RAM

Machine C → 8 GB RAM

Machine D → 8 GB RAM

You have **32 GB total physical RAM**, but this does **not automatically behave like one computer with 32 GB RAM**.

This is very important.

You cannot simply say:

8 GB + 8 GB + 8 GB + 8 GB



one Python process



32 GB RAM

Normally, each machine has its own memory.

Instead, you distribute the computation:

Machine A

RAM → 8 GB



process partition A

Machine B

RAM → 8 GB



process partition B

Machine C

RAM → 8 GB



process partition C

Machine D

RAM → 8 GB



process partition D

3. Your project is particularly suitable for partitioning

Suppose you have:

5 million thermal events

You don't need one machine to load:

5,000,000 × 48

rows.

Instead:

Machine 1 → events 1–1,000,000

Machine 2 → events 1,000,001–2,000,000

Machine 3 → events 2,000,001–3,000,000

Machine 4 → events 3,000,001–5,000,000

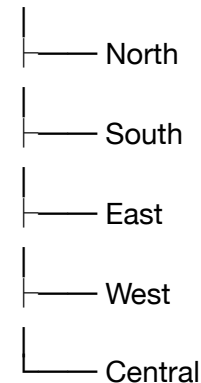
Each machine produces:
feature vectors
and you combine them afterward.

4. Geographic partitioning is even better

For your particular project, I would consider partitioning by **geographical region**.

For example:

India



Or by states/districts.

Then:

Worker 1

Maharashtra + Gujarat

Worker 2

Rajasthan + Madhya Pradesh

Worker 3

Tamil Nadu + Karnataka

Worker 4

West Bengal + Odisha

This is useful because your features are spatial.

For example:

distance_to_refinery

industrial_facility_count_3km

land_cover_fraction

can be processed regionally.

5. But be careful with temporal features

Your feature:

historical_frp_mean

means a worker cannot blindly look only at its own partition.

Suppose:

Worker 1 → Maharashtra

and an event is near the Maharashtra-Gujarat border.

Its historical data might include observations from both regions.

So your partitioning strategy needs to account for spatial neighborhoods.

A common approach is:

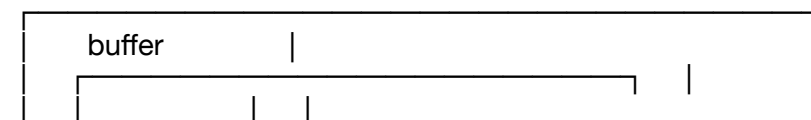
core region

+

buffer region

For example:

Worker 1





The buffer can support calculations such as:

3 km facility count

1 km land-cover fraction

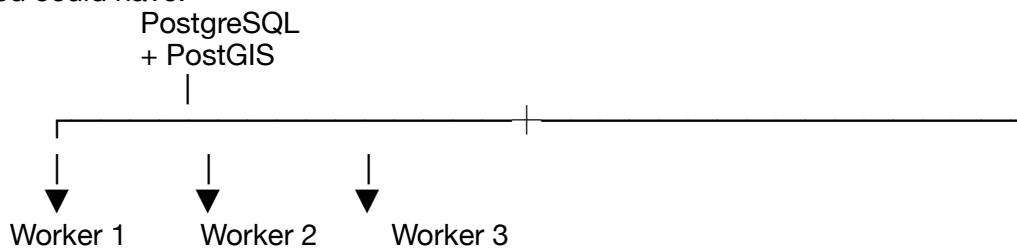
historical nearby detections

cluster calculations

while only the core-region events are written as that worker's final outputs.

6. Your PostgreSQL server can act as the central coordinator

You could have:



Each worker:

1. Gets its assigned events
2. Queries PostGIS
3. Extracts HLS features
4. Extracts ERA5 features
5. Calculates temporal features
6. Builds feature vectors
7. Writes results back

For example:

Worker 1

↓
feature_vectors
↓
PostgreSQL

Worker 2

↓
feature_vectors
↓
PostgreSQL

7. Don't let all machines write to the same rows

Give each event a unique ID:

event_id

and assign work using:

worker_id

For example:

job_id | event_id | worker_id | status

1	1001	W1	DONE
2	1002	W2	DONE
3	1003	W3	PROCESSING

This allows you to recover from a worker failure.

8. A simple architecture without complicated cluster software

If this is a college project, you don't necessarily need Kubernetes, Spark, Hadoop, etc.

You could have:

Laptop 1

Python Worker
8 GB RAM

Laptop 2
Python Worker
8 GB RAM

Laptop 3
Python Worker
8 GB RAM

Laptop 4
Python Worker
8 GB RAM



PostgreSQL/PostGIS

Each worker runs the same Python program with a different:
`worker_id`
and processes a different batch.
This is surprisingly effective.

9. Example

Suppose:

1,000,000 events

and you have four machines.

Divide them:

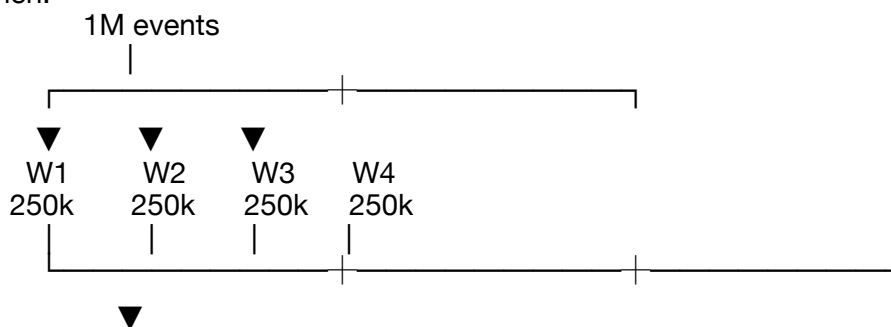
W1 → 1–250,000

W2 → 250,001–500,000

W3 → 500,001–750,000

W4 → 750,001–1,000,000

Then:



feature_vectors

Each machine only needs enough RAM for its current batch.

10. Process in chunks even on each machine

This is even more important.

Don't do:

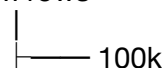
```
df = read_all_5_million_rows()
```

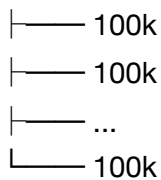
Instead:

```
for chunk in read_in_chunks():  
    process(chunk)  
    save(chunk)
```

Conceptually:

5M rows





So even an 8 GB machine can process a large dataset.

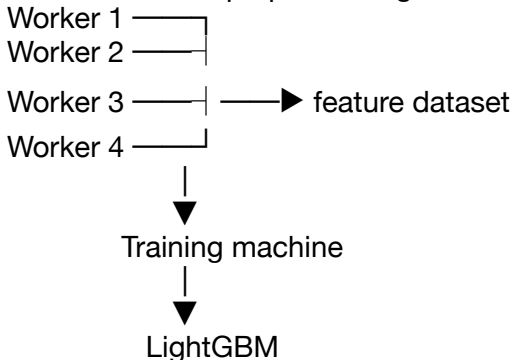
11. Then what about LightGBM itself?

There are three possibilities.

Case 1 — Dataset fits on one machine

This is the easiest.

After distributed preprocessing:



You might discover that your final feature dataset is small enough to train on a single 8/16 GB machine.

Remember:

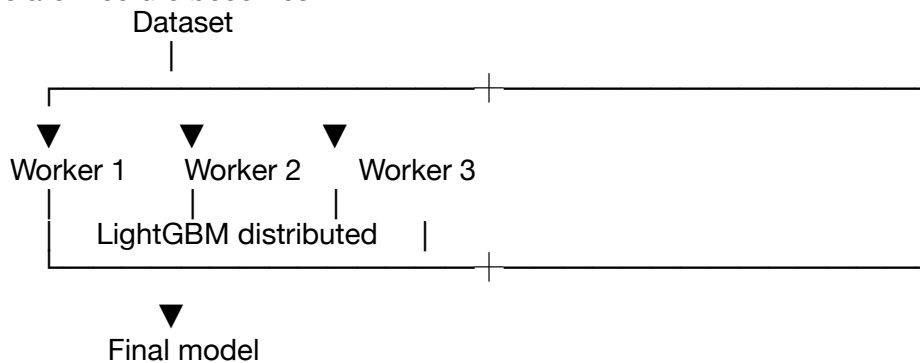
5 million × 48

is not necessarily enormous if stored efficiently.

12. Case 2 — Dataset doesn't fit on one machine

Then use distributed LightGBM.

The architecture becomes:



Each worker holds a partition of the training data.

The workers communicate during training to build the model collaboratively.

13. Why distributed training isn't simply "train three models"

This is a common misunderstanding.

This:

Machine 1 → train model A

Machine 2 → train model B

Machine 3 → train model C

doesn't automatically give you one distributed model.

You would have:

Model A

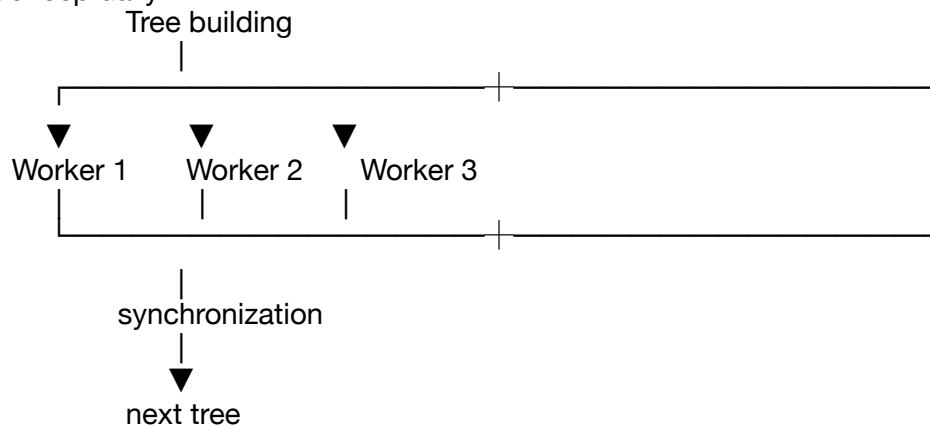
Model B

Model C

You'd then need an ensemble strategy.

Distributed LightGBM instead coordinates the workers during the training process.

Conceptually:



14. Network becomes important

When you distribute LightGBM training, machines have to communicate.

So:

CPU

RAM

are no longer the only concerns.

You also need:

network latency

network bandwidth

reliable connectivity

For machines in the same lab/local network:

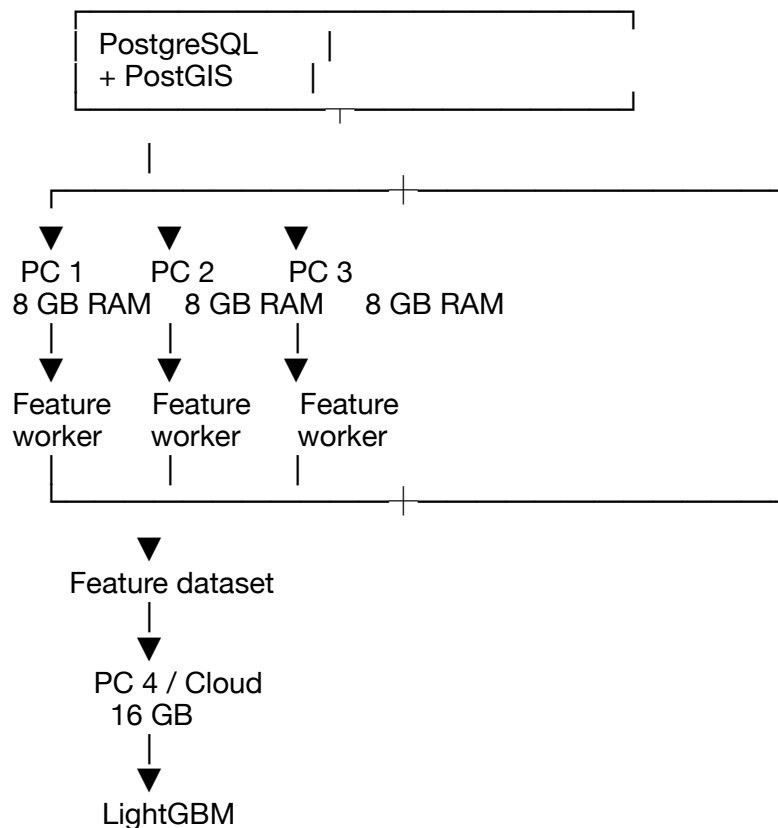
Ethernet

is preferable to unreliable Wi-Fi.

For your project, however, I'd avoid distributed LightGBM until you actually prove that a single machine cannot train your final feature dataset.

15. A better solution for your situation

If you have several weak machines, I'd use them like this:



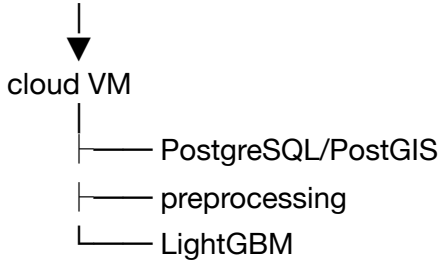
This avoids unnecessarily complicated distributed training.

16. You can also use cloud machines

If your local machines are insufficient, you can temporarily use cloud compute.

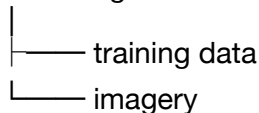
The pattern becomes:

Local development



Or:

Object storage



cloud workers

For a project/demo, you can scale the machines up only when training.

17. What I would NOT do

I wouldn't immediately build:

Kubernetes

+

Kafka

+

Spark

+

Hadoop

+

multiple GPU servers

just because you don't have 32 GB RAM.

That's a lot of infrastructure for your current model.

Your model is:

LightGBM

with:

~48 tabular features

So first use:

chunking

+

parallel preprocessing

+

efficient database queries

before introducing distributed ML infrastructure.

18. Your ideal low-RAM strategy

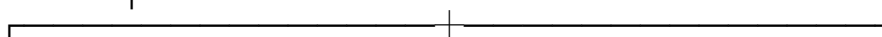
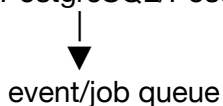
If you have, say:

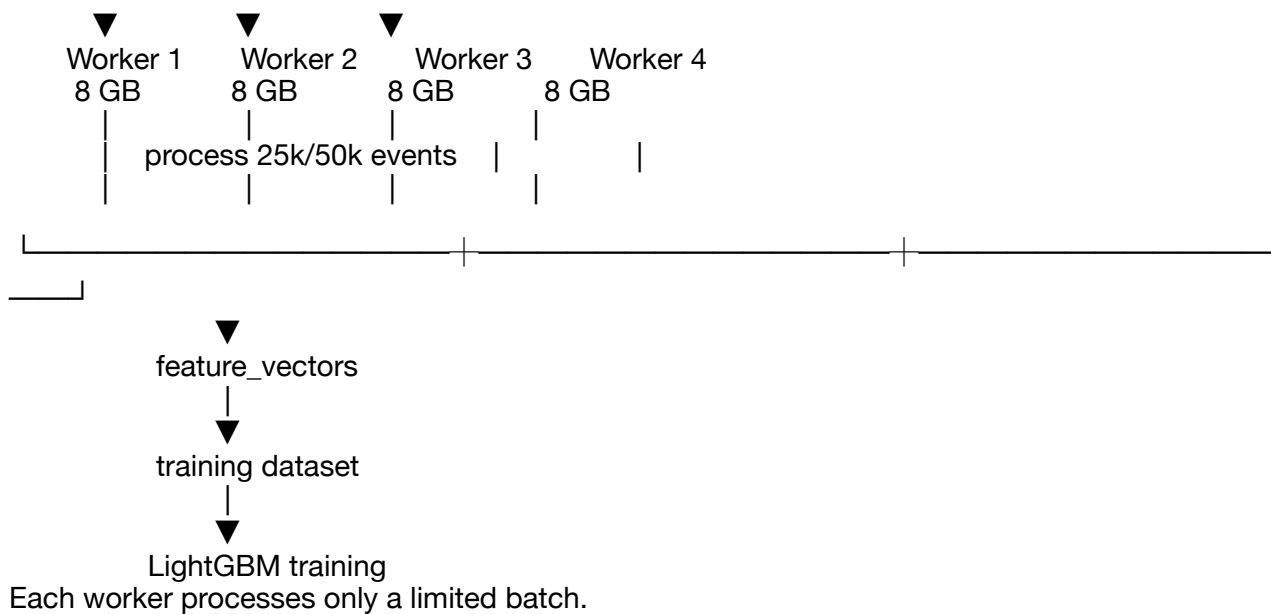
4 machines

8 GB RAM each

I'd implement:

PostgreSQL/PostGIS





19. One subtle issue: PostgreSQL can become your bottleneck

If four workers simultaneously ask PostGIS:

"find nearest refinery"

"find facilities within 3 km"

"calculate land-cover intersection"

millions of times, the database may become overloaded.

So you need:

PostGIS spatial indexes

+

batch queries

+

connection pooling

+

reasonable worker count

Don't assume:

4 workers = 4× faster

It might become:

4 workers

↓

PostGIS overloaded

↓

slower than 1 worker

20. Your spatial indexes become extremely important

For:

thermal_events.location

industrial_facilities.location

industrial polygons

land-cover geometries

use GiST spatial indexes.

For example:

```
CREATE INDEX idx_facilities_location
```

```
ON reference.industrial_facilities
```

```
USING GIST(location);
```

Without spatial indexing, a query can effectively become:

event

↓

compare against every facility

↓

compare against every facility

↓
 ...
 With spatial indexing:
 event
 ↓
 spatial index
 ↓
 nearby candidates
 ↓
 exact distance
 That can make an enormous difference.

21. Parallelism inside one machine also helps

You don't necessarily need multiple physical computers.

If your machine has:

4 CPU cores

8 GB RAM

you can use multiple CPU threads/processes while keeping each process's memory controlled.

For example:

CPU

|—— Worker 1

|—— Worker 2

|—— Worker 3

|—— Worker 4

But be careful with:

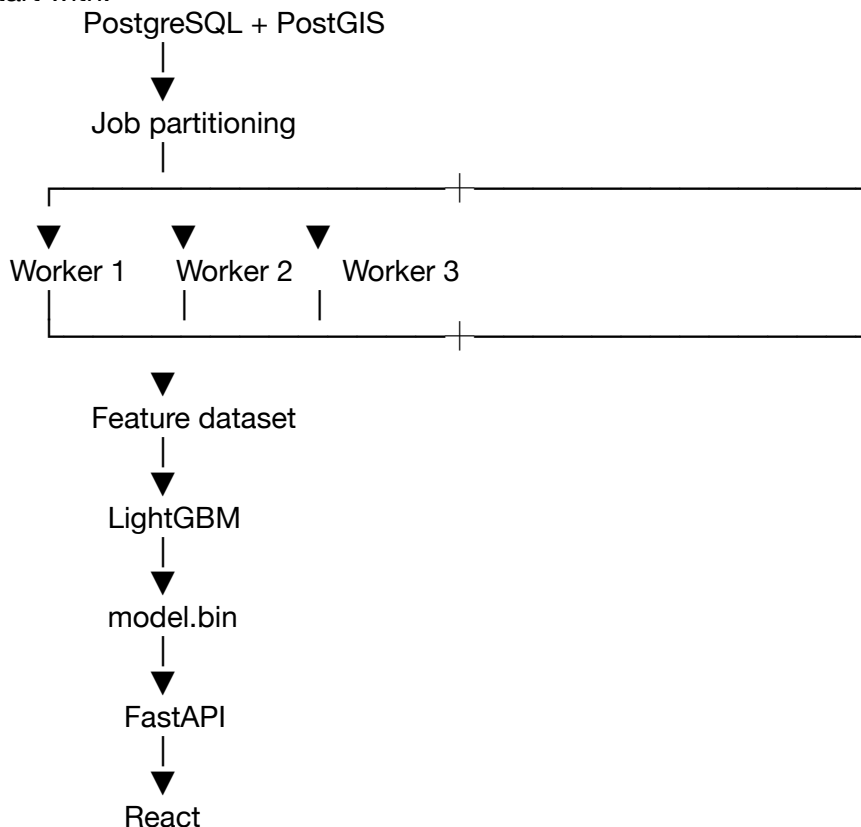
RAM = 8 GB

because four workers can each allocate memory.

So you need to control batch size.

22. The architecture I'd recommend for your project

Start with:



And make every worker:

stateless
as much as possible:
get batch
↓
process
↓
write results
↓
release memory
↓
get next batch

23. Most importantly: separate training from inference

Your deployed system doesn't need all those machines.

Training:

Multiple machines

↓
feature generation

↓
LightGBM training

↓
trained model

Deployment:

FastAPI

↓
feature vector

↓
LightGBM

↓
prediction

The deployed LightGBM model is comparatively lightweight.

You don't need to keep a whole training cluster running just to answer:

lat = X

lon = Y

Recommended plan for you

I'd build this in **three stages**:

Stage 1 — Single machine

8–16 GB RAM

↓
PostGIS

↓
feature generation

↓
LightGBM

Prove the complete pipeline works.

Stage 2 — Parallel feature generation

PC 1 —
PC 2 —

PC 3 — → PostgreSQL/PostGIS

PC 4 —

Use this when your data becomes large.

Stage 3 — Distributed LightGBM

Only if your final training dataset genuinely cannot be trained efficiently on one machine.



For your project, **Stage 2 is much more likely to give you the benefit you need than jumping immediately to Stage 3.**

If you tell me roughly **how many FIRMS records you expect (e.g. 1 lakh, 10 lakh, 1 crore) and how many machines you have + RAM/CPU of each**, I can design the exact parallel architecture—including how to partition the events, how workers claim jobs from PostgreSQL, batch sizes, and how the resulting feature vectors are merged without duplicating or losing events.